Code559

Front-End Developer Course

MODULE 12

Git, GitHub and Publishing

Move from a memorized routine to a workflow you understand: diff, branches, merges, Issues, pull requests and releases.

Free lesson: 559solutions.com/code559/module-12-git-github/

In this packet

1. **The lesson**
2. **Project: my-site through the full team workflow**
3. **Finishing work in Git**
4. **Troubleshooting: six common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn what your save routine actually does: staging, diffs, history, branches, merges, and Issues and pull requests connected with `Closes #1`. You also learn to tag a v1.0 release. It's what employers look for: a history of small, clear commits and merged pull requests shows you work the way a team does.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 5
- Core concepts 5
  - 1. Where does your work live in Git? 5
  - 2. What is `git status` telling you? 6
  - 3. What do `git add` and `git commit` actually do? 7
  - 4. How do you check your work before you commit? 8
  - 5. How do you read history, and what is a branch? 10
  - 6. How does a merge bring the branch back? 10
  - 7. What are `switch`, `restore` and `checkout`? 11
  - 8. How do you keep files out of Git? 12
  - 9. What goes in a README? 13
  - 10. How do you mark a finished version? 14
  - 11. Can VS Code do all of this with buttons? 15
  - 12. How do Issues and pull requests work together? 16
  - 13. What is GitHub Desktop? 19
  - 14. What does "dubious ownership" mean? 19
- Exercise 20
- Common misconception 23
- Summary 23
- Check your understanding 24
- Before next module 24
- Go further 25
- What to keep 25

Part 2: Project: my-site through the full team workflow 27

Part 3: Finishing work in Git 37

Part 4: Troubleshooting: six common problems 40

Part 5: Self-assessment 43

Part 6: Answers 44

About 559 Solutions 48

© 2026 559 Solutions LLC

PART 1

The lesson

Move from a memorized routine to a workflow you understand: diff, branches, merges, Issues, pull requests and releases.

Time: about 2–3 hours (estimate)

In this module you will learn:

- The Git mental model: working folder → staging area → commits → remote
- How to read `git status` and `git status --short`
- What `git add` and `git commit -m` really do, and why you make one change per commit
- How to check your work with `git diff`, `git diff --staged` and `git diff --stat` before every commit
- How to read history with `git log --oneline`
- How to make a branch with `git switch -c`, prove it is separate, merge it with `git merge` and delete it with `git branch -d`
- The checkout family: `git switch`, `git restore`, and `git checkout` as the older all-in-one
- How a `.gitignore` file keeps files out of Git, and how `git status` proves it
- How to write a `README.md` with four standard headings
- How the VS Code Source Control view does the same jobs as the terminal
- How GitHub Issues and pull requests connect through `Closes #1`, and how to leave line comments on your own pull request
- How to send a new branch to GitHub with `git push -u origin`, and bring a merged pull request back to your computer with `git pull`
- How to mark a version with `git tag v1.0`, and how a GitHub release is built on that tag
- What GitHub Desktop is (awareness only)
- What the `dubious ownership` message means and how to fix it

Why this matters

In Module 1 you learned five steps to save your site, in the "Save your work" box: `git status` → `git add .` → `git commit -m "…"` → `git push` → check the live page. If you did the packet projects, you have used them in every module since. That routine works. But it treats Git like a black box. You type the words and hope.

This module opens the box. You will see where your changes sit at each step, check them before they become history, and try a new page on a branch without risking the working site. Then you will use the GitHub tools that teams use every day: an Issue that names the work, a pull request that proposes it, and a release that marks a finished version.

© 2026 559 Solutions LLC

Employers read your GitHub profile. A history of small, clear commits, merged pull requests and a tagged v1.0 says "this person works the way we do." That is the goal of this module.

Before you start

1. Your `my-site` folder from earlier modules is on GitHub, and the live page loads.
2. In VS Code, open a terminal. Type `git --version` and `gh auth status`. Both should answer without an error (you set them up in Module 0). Then type `git config init.defaultBranch`. It should print `main` (Module 0, step 8). If it prints nothing, run `git config --global init.defaultBranch main` now. Without it, `git init` names the first branch `master`, and `git switch main` in the Exercise fails with "fatal: invalid reference: main".
3. This module's Exercise uses a practice folder called `practice-12`, inside your `code559` folder. **Not** inside `my-site`.

> **Checkpoint.** `git --version` prints a version, `git config init.defaultBranch` prints `main`, `gh auth status` says you are logged in, and you know where your practice folder will go.

A note on the pictures. Every terminal image in this lesson comes from a real run of Git 2.43.0 in a throwaway project on Linux, drawn in a Windows Terminal frame. The commit IDs (like `b4cf5d1`) are real for that run; yours will be different, because a commit ID depends on the time and the author too. GitHub.com, VS Code and GitHub Desktop pictures are drawings, because nothing in this lesson was pushed to GitHub. The `git push` and `git pull` steps in Concepts 10 and 12 were checked against a local repository standing in for GitHub.

Core concepts

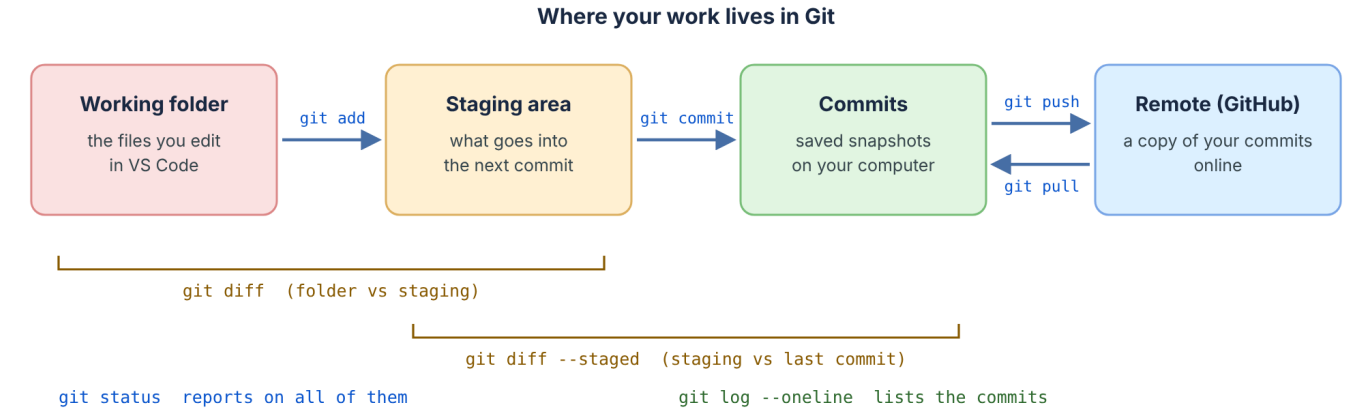
1. Where does your work live in Git?

Git keeps your project in four places. Most confusion about Git comes from not knowing which place a change is in.

The **working folder** is the files you see and edit. The **staging area** is a list of changes you have chosen for the next commit. **Commits** are saved snapshots, stored in the hidden `.git` folder on your computer. The **remote** is the copy on GitHub. `git add` moves a change from the folder to staging. `git`

© 2026 559 Solutions LLC

`commit` turns staging into a snapshot. `git push` sends your commits to GitHub. Nothing reaches GitHub until you push. `git pull` goes the other way: it downloads new commits from GitHub and brings your files up to date.



Each arrow is one command. `git diff` compares the folder with staging; `git diff --staged` compares staging with the last commit.

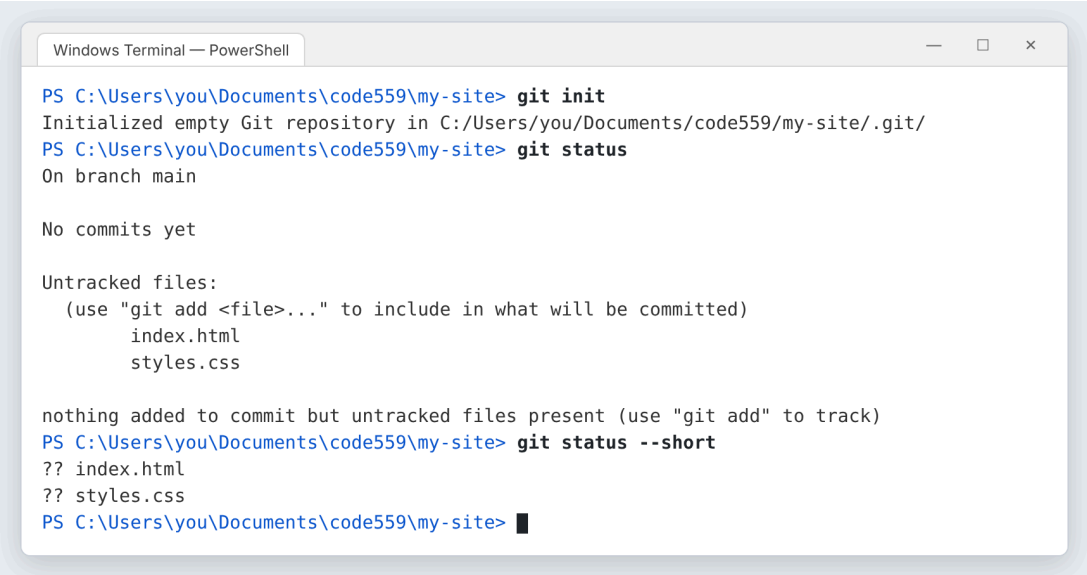
Predict: which box is a file in right after you save it in VS Code? (Answer: only the working folder.)

2. What is `git status` telling you?

`git status` is the question "where is everything right now?" Run it before and after every other Git command.

© 2026 559 Solutions LLC

In a brand-new project, Git lists your files as **untracked**: it sees them, but it is not keeping their history yet. `git status --short` gives the same facts in one line per file. The two characters at the start are two columns. The **left** column is the staging area; the **right** column is the working folder. `??` means untracked. `A` on the left means added to staging. `M` means modified: on the left it is staged, on the right it is not staged yet.



The long form explains; the short form is faster once you can read it. `??` means "untracked." (real output, rendered; captured on Linux, with the folder path changed to the course folder)

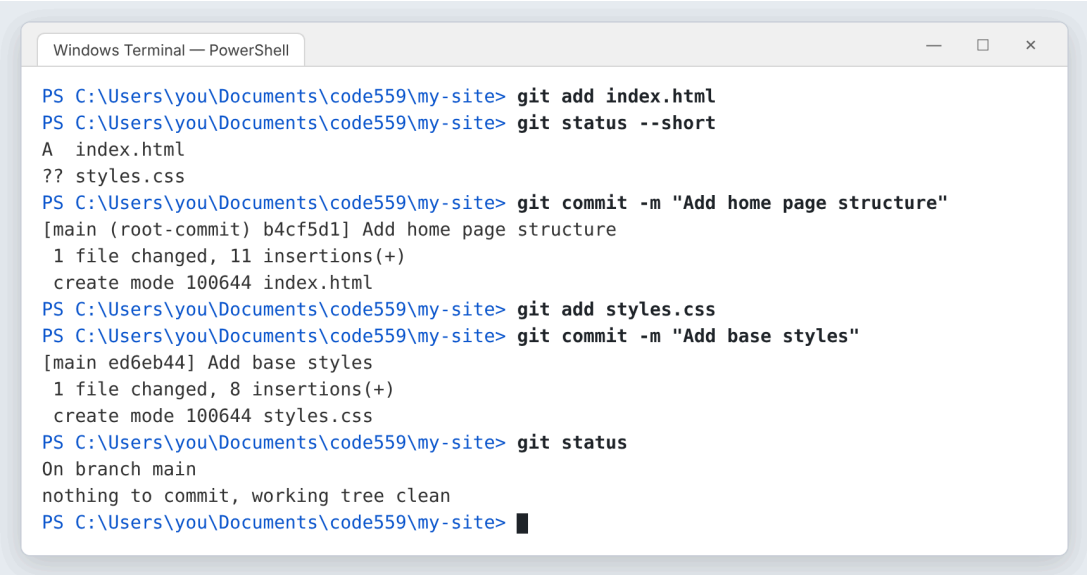
In your `my-site` folder, run `git status --short` now. An empty answer means nothing has changed since your last commit.

3. What do `git add` and `git commit` actually do?

`git add` chooses. `git commit` saves. They are separate so that you decide exactly what goes into each snapshot.

© 2026 559 Solutions LLC

You have used `git add .`, which stages everything. Naming one file, `git add index.html`, stages only that file. Then `git commit -m "message"` saves what is staged, and only that. This lets you follow the most useful habit in Git: **one change per commit**. A good message finishes the sentence "This commit will...": *Add home page structure*, *Add base styles*. Short, starts with a verb, says what changed. "stuff" and "fixes" tell your future self nothing.



After `git add index.html`, only that file has an `A`. Two files, two commits, two clear messages. "working tree clean" means the folder matches the last commit. (real output, rendered; captured on Linux with Git 2.43.0)

Look at your own `my-site` history with `git log --oneline` (Concept 5). Find one message that does not finish "This commit will...". You will write better ones from now on.

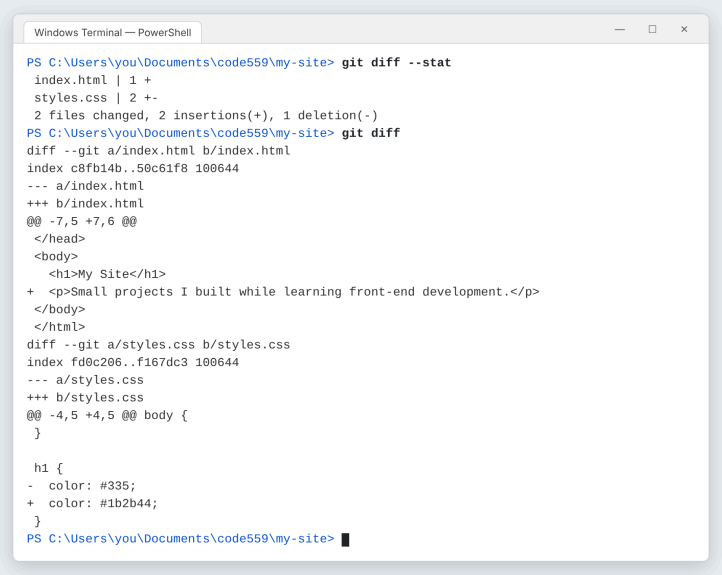
4. How do you check your work before you commit?

`git diff` shows exactly which lines changed. Reading it before every commit is how you catch a stray edit before it becomes history.

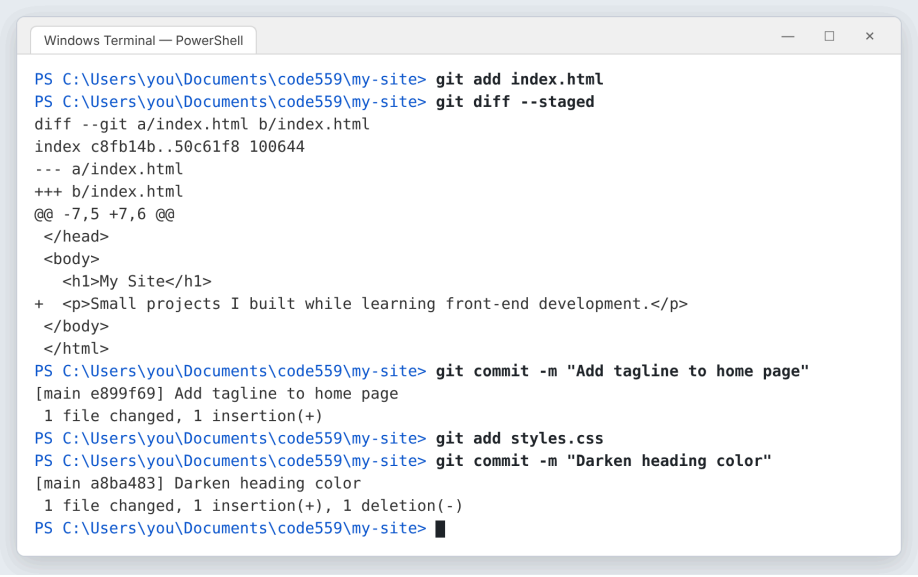
`git diff --stat` gives a summary: which files changed and how many lines. `git diff` shows the lines. A line starting with `+` was added; `-` was removed; the lines with no sign are context so you can find your place. Once you stage a file, plain `git diff` stops showing it, because it compares the folder with staging. To see what you are about to commit, use `git diff --staged`. If the output of `git diff` or `git log` is longer than your terminal window, Git shows it one screen at a time, with a `:` at the bottom. Press Space to see more, and press **q** to get back to the prompt.

© 2026 559 Solutions LLC

In the story below, two different changes are waiting: a new tagline in `index.html` and a darker heading color in `styles.css`. That is two changes, so it should be two commits.



`--stat` *first for the big picture, then the full diff. The color change shows as one* `-` *line and one* `+` *line. (real output, rendered; captured on Linux with Git 2.43.0)*



`git diff --staged` *shows only what the next commit will contain, so each commit holds one change. (real output, rendered; captured on Linux with Git 2.43.0)*

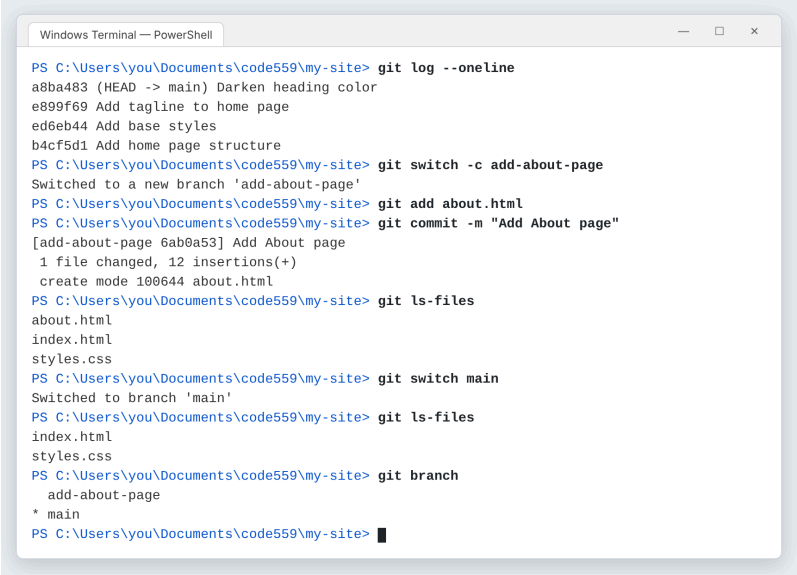
Before your next commit on `my-site`, run `git diff --staged` and read every `+` and `-` line. If a line surprises you, do not commit yet.

© 2026 559 Solutions LLC

5. How do you read history, and what is a branch?

`git log --oneline` lists your commits, newest first, one line each. A **branch** is a separate line of commits, so you can try something without touching `main`.

In the terminal, Git also writes labels in brackets beside some commits. `HEAD` means "where you are now," so `(HEAD -> main)` says you are on the branch `main`, at that commit. `git switch -c add-about-page` creates a branch and moves you onto it. Commits you make there belong to that branch only. To prove it, list the files Git is tracking with `git ls-files`, switch back with `git switch main`, and list them again. The new page is gone from `main`; it is safe on its branch. `git branch` lists your branches and puts a `*` beside the one you are on. The git-scm.com manual describes `-c` as "Create a new branch named *<new-branch>* ... before switching to the branch" (*git-switch*).



On the branch, `about.html` exists. Back on `main`, it does not. Same folder, different branch, different files. (real output, rendered; captured on Linux with Git 2.43.0)

Predict: if you open the folder in File Explorer while on `main`, will you see `about.html`? (No. Git removes it from the folder when you switch, and puts it back when you switch to the branch.)

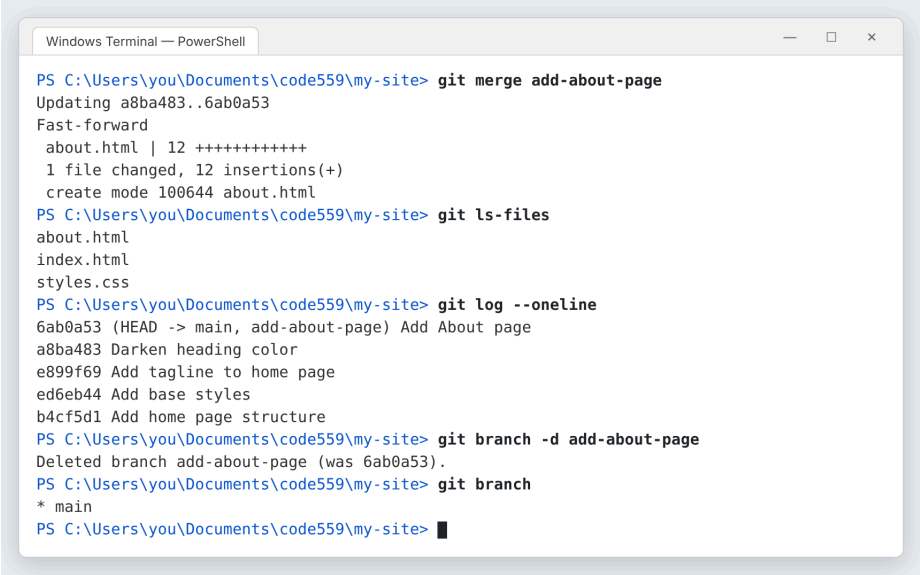
6. How does a merge bring the branch back?

When the branch is ready, you stand on `main` and run `git merge` with the branch name.

If `main` has not changed since you made the branch, Git does not need a new commit. It simply moves `main` forward to the branch's last commit. The manual calls this a **fast-forward**: "only update the branch pointer to match the merged branch; do not create a merge commit" (*git-merge*). After the merge, the branch has done its job; `git branch -d` deletes the name. Your commits stay. Git refuses to delete a branch whose work is not merged yet. If you run `git branch -d add-about-page` before

© 2026 559 Solutions LLC

the merge, Git stops with "error: the branch 'add-about-page' is not fully merged". The next line suggests `git branch -D`. Do not use `-D` unless you want to throw that work away: it deletes the branch "irrespective of its merged status" (*git-branch*).



"Fast-forward" means `main` just moved ahead; the log shows no extra merge commit. (real output, rendered; captured on Linux with Git 2.43.0)

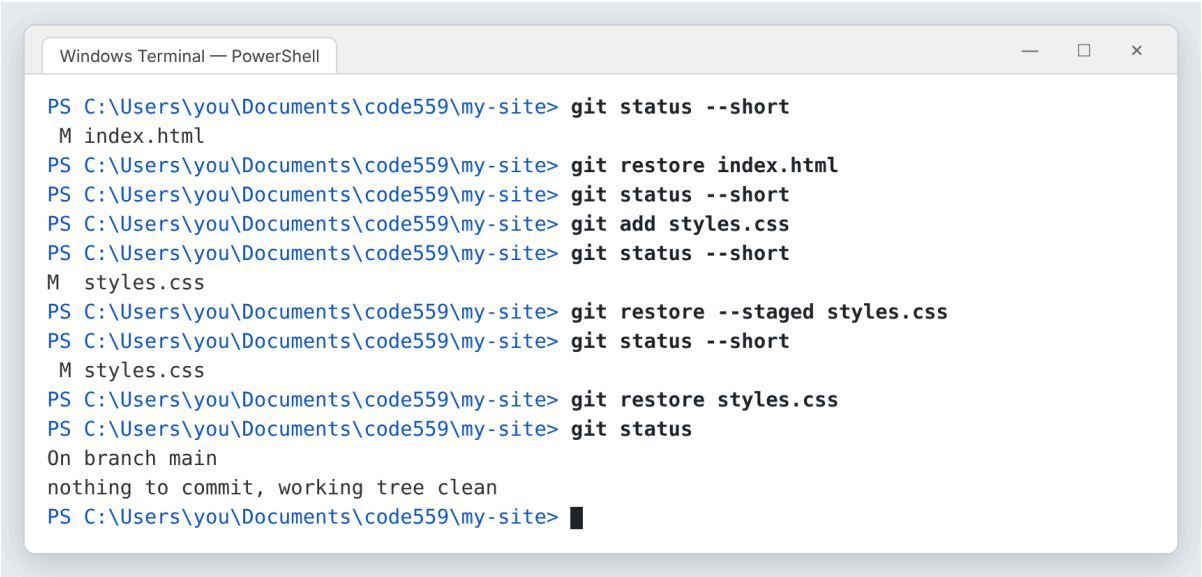
Explain in one sentence why the log after the merge has exactly five lines, not six.

7. What are `switch`, `restore` and `checkout`?

These three commands overlap, and you will see all three in tutorials.

`git switch` changes branches. `git restore` puts a file back. `git restore index.html` throws away the changes you have not staged: it copies the file back from the staging area. If you staged nothing, that is the same as the last commit. The manual says the contents are restored "from the index" (the staging area) unless you add `--staged` (*git-restore*). `git restore --staged styles.css` takes a file out of staging but keeps your edit in the folder. `git checkout` is the older command that does both jobs; its manual title is "Switch branches or restore working tree files" (*git-checkout*). `git`

`checkout -b name` is the older way to write `git switch -c name`. You will meet it in older tutorials; this course uses `switch` and `restore` because each does one job. One more word you may hear is "check-in." That is not a Git command.



Watch the `M` move from the left column (staged) to the right column (not staged) after `git restore --staged`. (real output, rendered; captured on Linux with Git 2.43.0)

Be careful: `git restore` on a file throws away your edits to it. There is no undo for edits you never committed. Run `git diff` on the file first.

8. How do you keep files out of Git?

Some files in your folder should never be committed: editor settings, system files, logs.

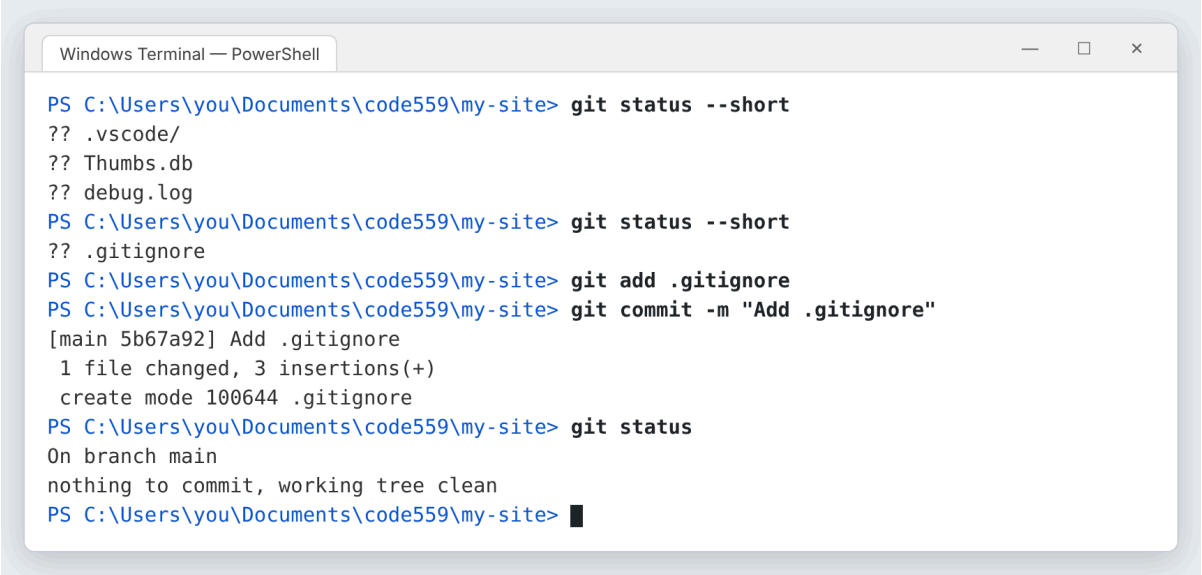
A file named `.gitignore`, in the top of your project, lists patterns Git should ignore. Use this starter:

```
.vscode/
Thumbs.db
*.log
```

`.vscode/` (with the slash) matches a folder; the manual says a pattern ending in a slash "will only match directories." `Thumbs.db` is a file Windows can create to store picture thumbnails. `*.log` matches every file ending in `.log`, because "an asterisk `*` matches anything except a slash" (*gitignore*). One catch: "Files already tracked by Git are not affected" (*gitignore*). Add the `.gitignore` before you commit those files, not after. If you already committed one, the same page says to use `git`

© 2026 559 Solutions LLC

`rm --cached` "to remove the file from the index" (the staging area). For the editor folder, type `git rm -r --cached .vscode` (`-r` takes the whole folder), then commit. The files stay in your folder; Git stops tracking them.



The proof: three untracked entries disappear the moment `.gitignore` exists. The files are still in the folder; Git just stops reporting them. (real output, rendered; captured on Linux with Git 2.43.0)

Run `git status --short` in `my-site`. If you see `.vscode/` or a `.log` file listed, that is your sign to add a `.gitignore`. If `git ls-files` lists a file inside `.vscode/`, it was committed before; use `git rm -r --cached .vscode` as above.

9. What goes in a README?

`README.md` is the first thing a visitor sees on your repository page. GitHub shows it below the file list.

The `.md` means Markdown: `#` makes a heading, `##` a smaller one. This course uses four headings for every project: **What it does**, **Built with**, **What I learned**, **Live site**. Keep each to a sentence or two. "What I learned" is the one employers read.

This is the README committed in the story (the commit appears in Concept 10):

```
# My Site

## What it does
A small personal website with a Home page and an About page.

## Built with
```


© 2026 559 Solutions LLC

```
HTML and CSS.

## What I learned
How to keep a project's history with Git: one change per commit,
branches for new pages, and a .gitignore for files that do not belong.

## Live site
https://your-username.github.io/my-site/
```

Draft the "What I learned" section for your own `my-site` in two sentences. You will use it in the Project.

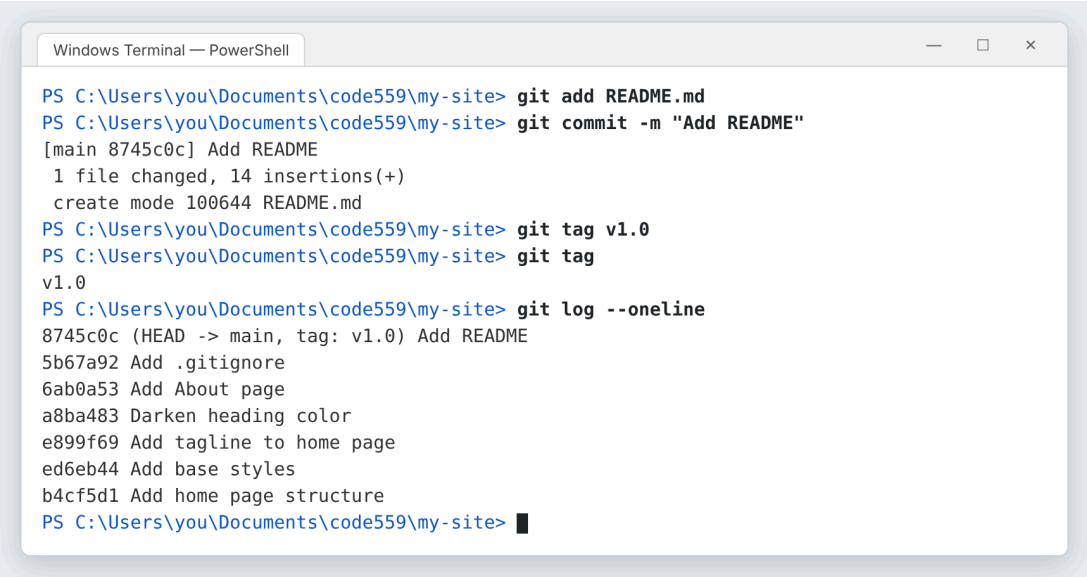
10. How do you mark a finished version?

A **tag** is a name stuck to one commit, such as `v1.0`. A GitHub **release** is a page built on a tag.

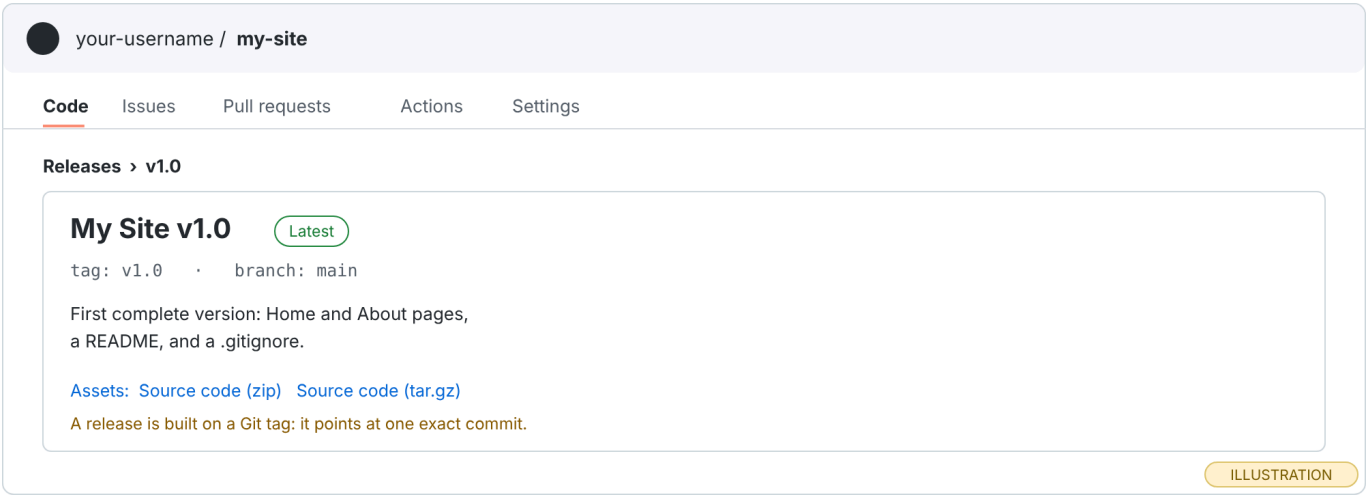
`git tag v1.0` puts the name on the commit you are on, so check that it is the right one first. If your last change was merged on GitHub, run `git switch main` and `git pull` (Concept 12), then check `git log --oneline`, and only then tag. `git tag` lists your tags, and `git log --oneline` shows the tag next to the commit as `tag: v1.0`. GitHub Docs explains the link: "Releases are based on Git tags, which mark a specific point in your repository's history" (*About releases*). A plain `git push` does not send tags. You send one with `git push origin v1.0`, and Git answers with a line like `* [new tag] v1.0 -> v1.0`. Or you can let GitHub create the tag when you draft the release on the website (*Managing releases in a repository*).

© 2026 559 Solutions LLC

Your release, and the form on GitHub that makes it, may look a little different from the picture below. In the form, look for the **Choose a tag** menu, the **Release title** field and the **Publish release** button; those names come from GitHub Docs.



`tag: v1.0` now sits on the README commit. That exact snapshot can always be found again by name. (real output, rendered; captured on Linux with Git 2.43.0)



What a release looks like on GitHub: a title, notes and downloads, all pointing at the tagged commit.

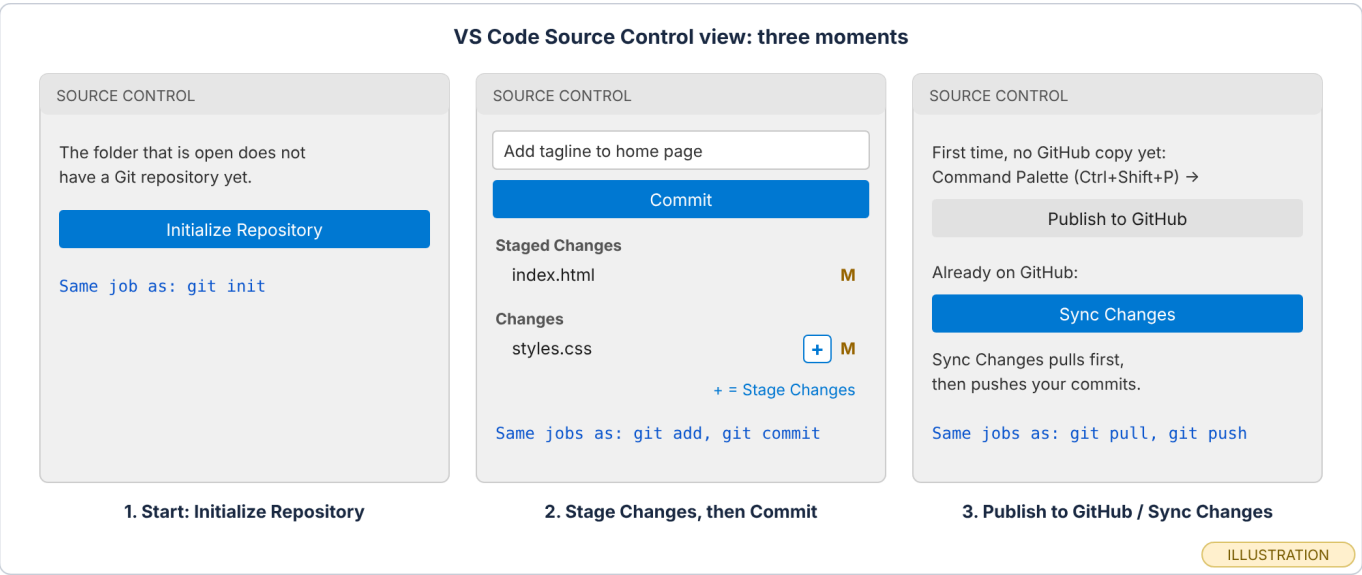
Run `git tag` in `my-site`. You probably have none yet. You will practice tagging with `v0.1` in the Exercise (step 8); if you do the packet project, you add `v1.0` there.

11. Can VS Code do all of this with buttons?

Yes. The Source Control view in VS Code runs the same Git commands behind its buttons.

© 2026 559 Solutions LLC

In a folder with no repository, the view shows **Initialize Repository** (the same as `git init`). When you have changes, each file has a **+** button, **Stage Changes**, which is `git add`. You type a message in the box and select **Commit**. To put a brand-new repository on GitHub you open the Command Palette (Ctrl+Shift+P) and run **Publish to GitHub**. After that, **Sync Changes** sends and receives commits; VS Code's documentation says it "pulls remote changes before pushing your commits" (*Quickstart: use source control in VS Code*). The terminal is still the main tool in this course, because its messages teach you what happened. Use the buttons once you can say which command each one runs.



Each button is a Git command you already know.

Open the Source Control view in `my-site` and match each file it lists to a line of `git status --short`.

12. How do Issues and pull requests work together?

An **Issue** is a note on GitHub that names a piece of work. A **pull request** (PR) proposes the commits that do that work. GitHub Docs puts it plainly: "Pull requests are proposals to merge code changes into a project" (*Pull requests*).

The flow is: open an Issue (say, #1 "Add a Links page"); make a branch with `git switch -c add-links-page`; commit a new `links.html` on it; push the branch; open a pull request from the branch into `main`. The pull request gets the number #2, because Issues and pull requests in one repository share one list of numbers ("Pull requests are a type of issue," *REST API endpoints for pull requests*). In the PR description, write **`Closes #1`**. When the PR is merged, GitHub closes Issue #1 for you. The keywords that work are close, closes, closed, fix, fixes, fixed, resolve, resolves and resolved, and GitHub only reads them "when the pull request targets the repository's *default* branch" (*Linking a pull request to an issue*). For you, that is `main`.

© 2026 559 Solutions LLC

Pushing a brand-new branch takes a little more than your usual `git push`:

```
git push -u origin add-links-page
```

`origin` is the name your GitHub copy got in Module 1 (the `--remote=origin` part of `gh repo create`). `-u` links your branch to a branch of the same name on GitHub; the manual says it adds an "upstream (tracking) reference" (*git-push*). After that, a plain `git push` on this branch works. If you forget `-u`, a plain `git push` stops with "The current branch add-links-page has no upstream branch" and prints the command to use (`--set-upstream` is the long form of `-u`).

Working alone, you can still review your own work. In the PR, open **Files changed**, hover a line number, and click the blue comment icon. Type a note, then choose **Comment** for one note or **Start a review** for several (*Commenting on a pull request*). If you start a review, finish it: click **Submit review** (some screens call this first button **Review changes**), write a short summary, and click **Submit review** again. Until then, your notes are "pending and only visible to you" (*Reviewing proposed changes in a pull request*). Reading your own diff on GitHub, slowly, catches mistakes you skipped past in the terminal.

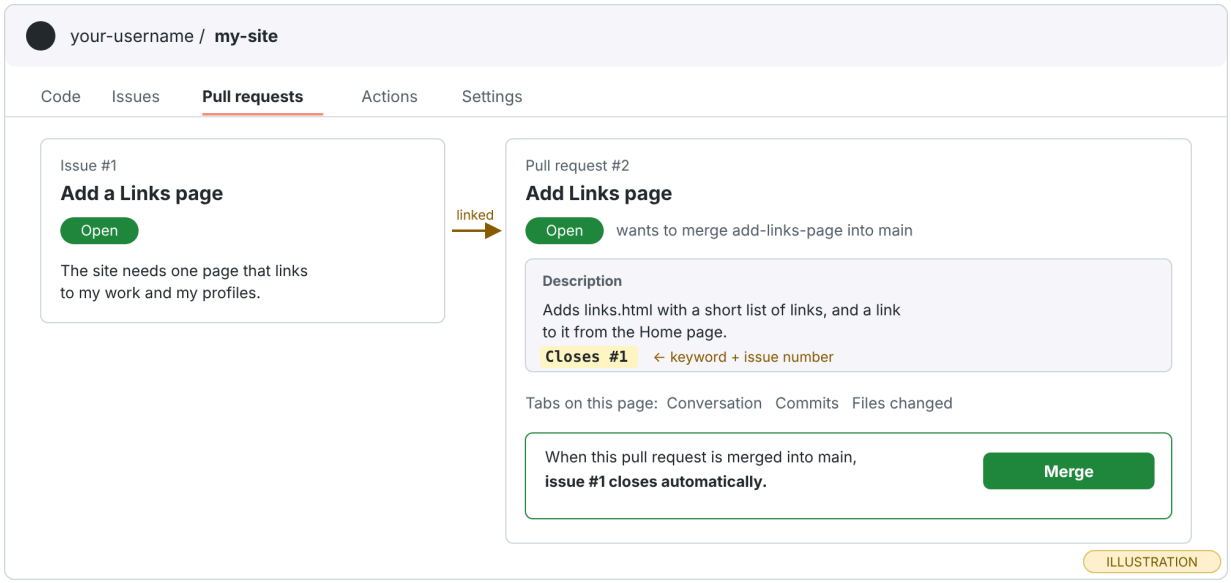
When you merge the pull request on GitHub, the merge exists only on GitHub, not on your computer yet. To bring it home, run:

```
git switch main
git pull
git log --oneline
```

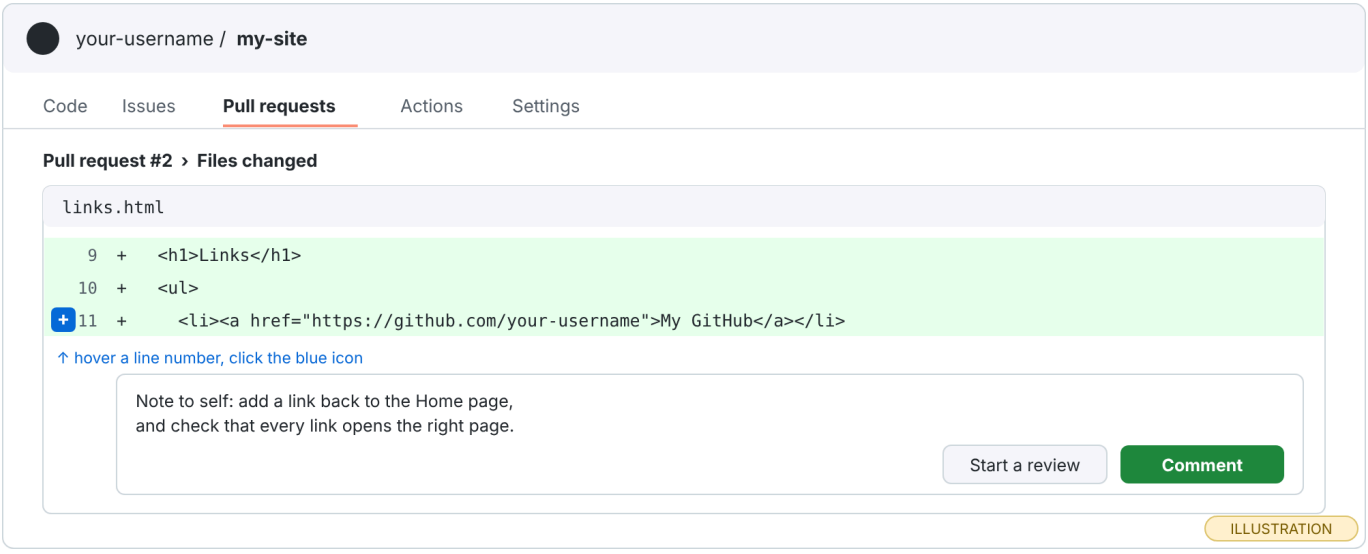
`git switch main` may say your branch is "up to date"; it has not asked GitHub yet, so pull anyway. `git pull` downloads the new commits and moves your `main` forward; look for `Fast-forward`. `git log --oneline` should now show GitHub's merge commit ("Merge pull request #2 ...") at the top,

© 2026 559 Solutions LLC

marked `(HEAD -> main, origin/main)`; Git 2.48 and later may also add `origin/HEAD` there. Only now tag the version (Concept 10). A tag made before `git pull` lands on an older commit, without the new page.



`Closes #1` in the description links the two. Merge into `main`, and the Issue closes.



A line comment on your own pull request: a note pinned to the exact line it is about.

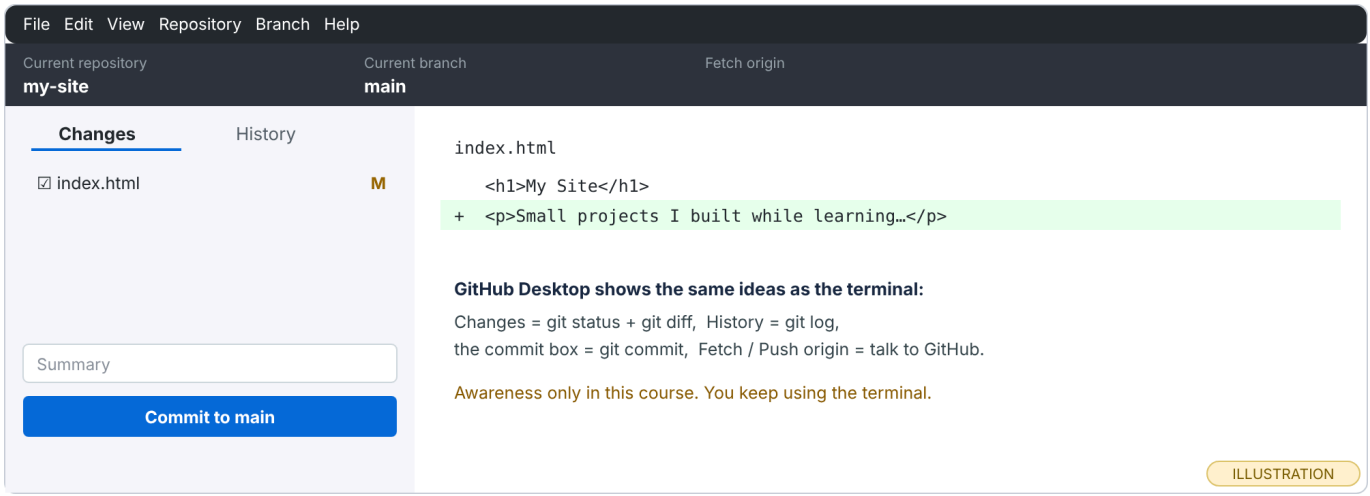
Write, on paper, the title of an Issue for the next change you want on `my-site`. Keep it to a few words that start with a verb.

© 2026 559 Solutions LLC

13. What is GitHub Desktop?

GitHub Desktop is "a free, open source application that helps you to work with files hosted on GitHub or other Git hosting services" (*About GitHub Desktop*). It runs on Windows and macOS.

It shows the same ideas you now know: a Changes list (status and diff), a History list (log), a commit box, and buttons to talk to GitHub. In this course it is **awareness only**. You do not need to install it. If a future job uses it, you will recognize every part.



Different screen, same model: changes, staging, commit, history.

Name the terminal command that matches the History tab. (`git log`.)

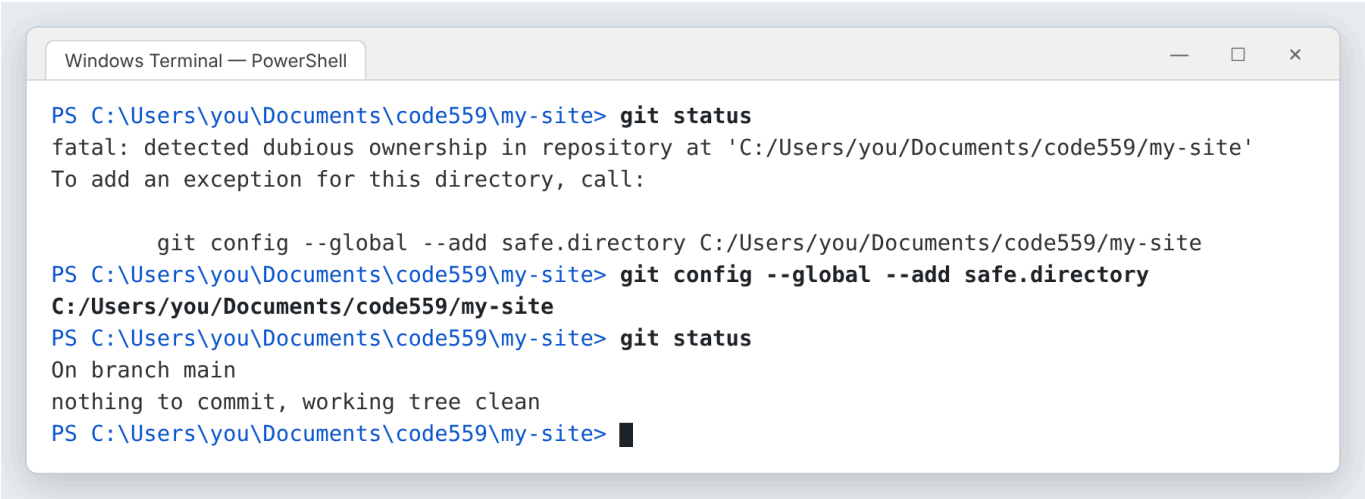
14. What does "dubious ownership" mean?

One day Git may refuse to work in a folder and print `fatal: detected dubious ownership in repository`.

Git checks that the folder belongs to the user account running Git. If it does not, Git stops, so that a repository someone else controls cannot run settings on your account. On Windows this tends to happen with folders on an external or network drive, or folders copied from another account. Your work is fine. Git prints the fix itself: one `git config --global --add safe.directory …` command

© 2026 559 Solutions LLC

with your folder's path. Copy it exactly as Git prints it, run it, and run `git status` again. On Windows, Git for Windows also prints which account owns the folder and which account you are using, so your message has a few more lines than the picture; the last line is still the fix.



Git names the exact command to run. After it, `git status` works. The message was caused by giving a copy of the project to a different user account. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Which folders trigger this message depends on your drives and accounts, so you may never see it. If you do, your folder path will be in the message instead of the one in the picture.

If you ever see this message, read the last line first. That is the fix.

Exercise

Your hands, about 45 minutes (estimate). Type every command. Do not paste. This is a throwaway folder: you will not push it.

1. **Make the practice folder.** In File Explorer, create `practice-12` inside your `code559` folder (beside `my-site`, not inside it). Open it in VS Code with **File → Open Folder...**, then **Terminal → New Terminal**. Check that the prompt ends in `practice-12>`.

2. **Start a repository.** Type `git init`. Create a file `notes.txt` with one line, `Favorite color: blue`. Press **Enter** at the end of the line, so the cursor sits on an empty line 2. Save it. Then:

```
git status --short
git add notes.txt
git commit -m "Add notes"
```


© 2026 559 Solutions LLC

Predict before you press Enter on the first line: what two characters will start the line? (`??`.)

3. **Make a branch.** `git switch -c add-hobby`. Add a second line to `notes.txt`, for example `Hobby: drawing`, and press **Enter** at the end of it too. Save. Then check before you commit:

   ```
   git diff
   git add notes.txt
   git diff --staged
   git commit -m "Add hobby line"
   ```

   Both diffs should show one `+` line. If you also see a `-` line with your color, or the note `\ No newline at end of file`, a line in the file did not end with Enter. Git counts the line ending as part of the line, so that line shows as changed. It is not an error: carry on, and end each line with Enter from now on.

4. **Prove the branch is separate.** `git switch main`. Look at `notes.txt` in VS Code (or type `cat notes.txt`). The hobby line is gone.

5. **Merge it.** Still on `main`: `git merge add-hobby`. Look for the word `Fast-forward`. Then `git branch -d add-hobby` and `git log --oneline`. You should see two commits.

6. **Break it on purpose: a merge conflict.** You will change the same line two different ways on two branches.

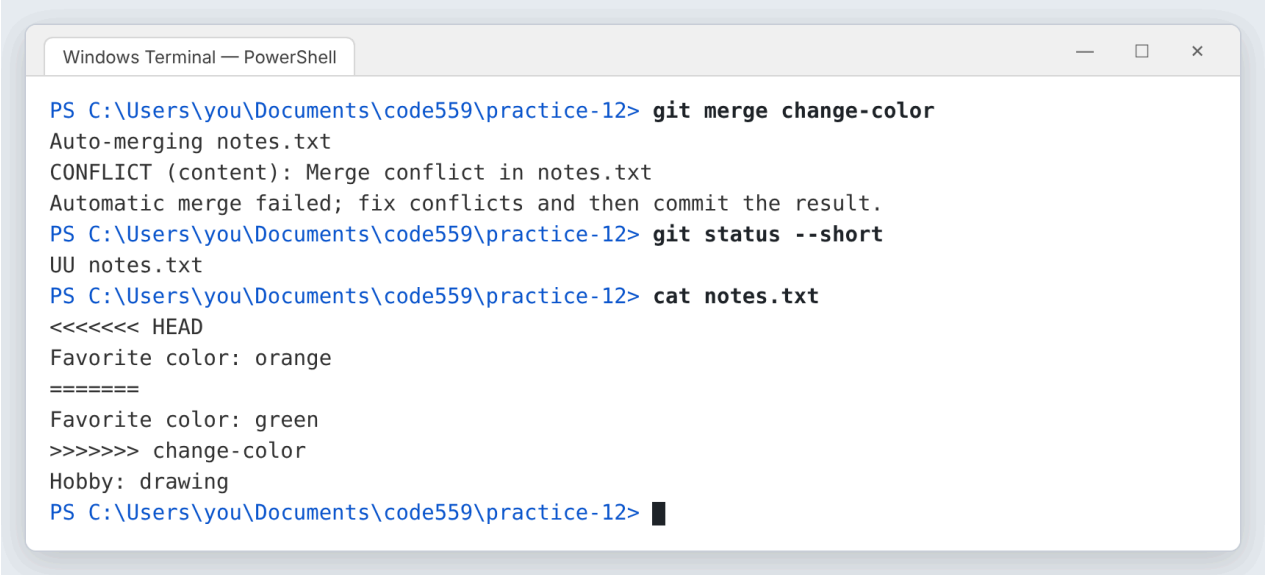
   ```
   git switch -c change-color
   ```

   Change `blue` to `green` in `notes.txt`, save, then `git add notes.txt` and `git commit -m "Change color to green"`. Now `git switch main`, change `blue` to `orange`, save, `git add notes.txt`, `git commit -m "Change color to orange"`. Finally:

   ```
   git merge change-color
   ```

   Read the whole message.

© 2026 559 Solutions LLC

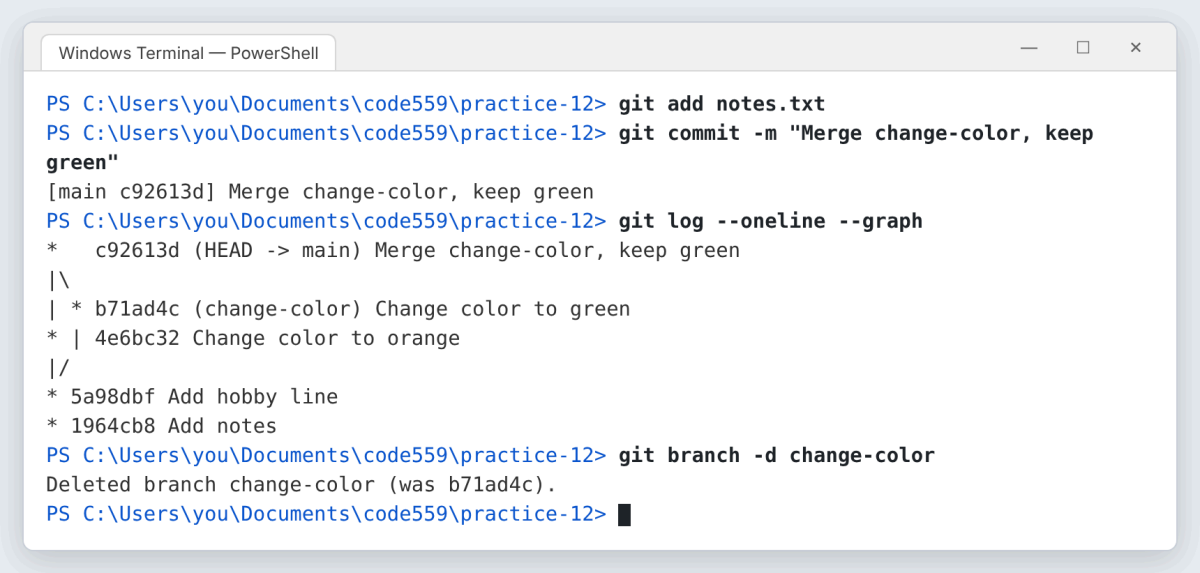


Git merged what it could (the hobby line) and stopped at the line both branches changed. `UU` means "both changed it." Between `<<<<<<< HEAD` and `=======` is your side (`main`); below it, the branch. (real output, rendered; captured on Linux with Git 2.43.0. On Windows, `cat` in PowerShell shows the same file lines.)

7. **Fix it.** Open `notes.txt` in VS Code. Delete the three marker lines (`<<<<<<<`, `=======`, `>>>>>>>`) and the color you do not want, so one `Favorite color` line is left. VS Code can do this for you with the links it shows above the conflict: **Accept Current Change** keeps your side on `main` (the line `Favorite color: orange`) and **Accept Incoming Change** keeps the branch's side (the line `Favorite color: green`) (*Resolve merge conflicts in VS Code*). Save. Then:

```
git add notes.txt
git commit -m "Merge change-color, keep green"
git log --oneline --graph
git branch -d change-color
```


© 2026 559 Solutions LLC



This time the log shows a real merge commit with two parents: the graph splits and joins. Compare it with the straight line after the fast-forward in step 5. (real output, rendered; captured on Linux with Git 2.43.0)

8. **Tag it.** `git tag v0.1`, then `git log --oneline -1`. The newest commit should show `tag: v0.1`.

9. **Modify.** Make one more branch of your own, change one line, save, and commit it on the branch (`git add notes.txt`, then `git commit -m "…"`). Then `git switch main` and merge your branch. Before you merge, think this through: will it be a fast-forward, and why? If you also committed something on `main`, the merge makes a merge commit, and Git opens an editor in the terminal for its message. Git for Windows uses Vim unless you chose another editor. To keep Git's message, press **Esc**, type `:wq` and press **Enter**.

Common misconception

> *"When I commit, my work is saved on GitHub."*

What would you say? Hint: look at the mental-model picture in Concept 1. Which arrow leads from the Commits box to GitHub?

Summary

Git keeps your work in four places, and `git add`, `git commit` and `git push` move it from the working folder to staging, to commits, to GitHub. `git status --short` tells you where each file is, with the left column for staging and the right column for the folder. One change per commit, with a message that

© 2026 559 Solutions LLC

finishes "This commit will...", makes your history readable. `git diff`, `git diff --staged` and `git diff --stat` let you check every line before it becomes history, and `git log --oneline` reads that history back. A branch made with `git switch -c` keeps new work separate until `git merge` brings it in, often as a fast-forward, and `git branch -d` tidies up. `git switch` changes branches and `git restore` puts files back; `git checkout` is the older command that does both. A `.gitignore` keeps editor, system and log files out, and `git status` proves it. A README with four headings tells visitors what the project is. A tag names a version, and a GitHub release is built on it. VS Code's Source Control view and GitHub Desktop do the same jobs with buttons. Issues name the work, pull requests propose it, `Closes #n` links them, and line comments let you review yourself. A new branch goes to GitHub with `git push -u origin` and the branch name, and after a pull request is merged there, `git pull` on `main` brings the merge to your computer before you tag. "Dubious ownership" is a safety check, and Git prints its own fix.

Check your understanding

1. `git status --short` shows these two lines. Which file is ready to commit, and how can you tell?

   ```
   M  styles.css
    M index.html
   ```

2. In the Exercise, why did step 5 say `Fast-forward` while step 6 stopped with a conflict?

3. **What would you test** to prove that your `.gitignore` is working?

Before next module

This module is the full workflow, so instead of the short routine, check off the whole thing:

- [ ] I can draw the four places (folder, staging, commits, remote) and name the command between each
- [ ] I run `git status` (or `--short`) before and after other Git commands, and can read the two columns
- [ ] I run `git diff --stat`, `git diff` and `git diff --staged` before every commit
- [ ] Each commit holds one change, with a message that finishes "This commit will..."
- [ ] I can make a branch with `git switch -c`, prove it is separate, merge it, and delete it
- [ ] I can fix a merge conflict by removing the markers, then `git add` and `git commit`
- [ ] I know to run `git pull` on `main` after a pull request is merged, and to tag only after that
- [ ] I can still do the short routine: `git status` → `git add .` → `git commit -m "…"` → `git push` → check the live page

If you did this module's packet project in `my-site`:

© 2026 559 Solutions LLC

- [ ] `my-site` has a `.gitignore` (`.vscode/`, `Thumbs.db`, `*.log`) and a `README.md` with the four headings
- [ ] I have opened an Issue, a pull request with `Closes #n`, and left a line comment on my own PR
- [ ] After the pull request was merged, I ran `git pull` on `main` before tagging
- [ ] `my-site` has a `v1.0` tag and a GitHub release

Go further

- git-scm.com manual pages: *git-switch*, *git-merge* (the "Fast-forward merge" and "How conflicts are presented" sections), *gitignore*, *git-restore*, *git-checkout*
- GitHub Docs: *Pull requests*, *Linking a pull request to an issue*, *Commenting on a pull request*, *About issues*, *About releases*, *About GitHub Desktop*
- VS Code Docs: *Quickstart: use source control in VS Code* and *Source control in VS Code*
- GitHub Skills, free practice exercises that run inside a repository of your own:
  - *Communicate using Markdown* (`skills/communicate-using-markdown`) for your README
  - *Review pull requests* (`skills/review-pull-requests`)
  - *Resolve Merge Conflicts* (`skills/resolve-merge-conflicts`)
  - *Create a release based workflow* (`skills/release-based-workflow`)

What to keep

Keep your `practice-12` folder until you finish the Project; its `git log --oneline --graph` is a picture of both kinds of merge. Keep your Issue title and your "What I learned" draft. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: git-scm.com "git-switch", "git-merge", "gitignore", "git-restore", "git-checkout"; GitHub Docs "Pull requests" (the page at the about-pull-requests address), "Linking a pull request to an issue", "Commenting on a pull request", "About issues", "About releases", "About GitHub Desktop"; VS Code Docs "Quickstart: use source control in VS Code" and "Source control in VS Code"; GitHub repositories skills/communicate-using-markdown, skills/review-pull-requests, skills/resolve-merge-conflicts, skills/release-based-workflow. Checked 2026-10-11: git-scm.com "git-restore" (default source), "git-push" (`--set-upstream`), "git-log" (`--decorate`, default `auto`); Git documentation `core.pager` (Git sets `LESS` to `FRX`); Git source code, setup.c and compat/mingw.c (the dubious-ownership message, and the owner lines Git for Windows adds); GitHub Docs "Managing releases in a repository" and "Configuring commit merging for pull requests"; Git 2.48.0 release notes (`git fetch` creates a missing `origin/HEAD`); git-scm.com "gitignore" (NOTES: `git rm --cached`), "git-rm" (`-r`, `--cached`), "git-branch" (`-D`, `--force`); Git source code, builtin/branch.c (the not-fully-

© 2026 559 Solutions LLC

merged message); GitHub Docs "About GitHub Desktop" (full sentence), "Commenting on a pull request" and "Reviewing proposed changes in a pull request" (Submit review, pending comments), "REST API endpoints for pull requests" ("Pull requests are a type of issue"); VS Code Docs "Resolve merge conflicts in VS Code" (Accept Current Change, Accept Incoming Change; current and incoming in a merge); Git for Windows installer source (git-for-windows/build-extra on GitHub, `installer/install.iss`*: Vim is the default editor).*

© 2026 559 Solutions LLC

PART 2

Project: my-site through the full team workflow

Your hands, about 2 hours (estimate). In the Exercise you used branches, merges and a tag in a throwaway folder. Now you use them on your real site, the way a team does. You add a `.gitignore` and a README. Then you make one real improvement, a link to your site's code in the footer of every page. It goes all the way through: an Issue, a branch, a pull request, your own review, a merge, `git pull`, a `v1.0` tag and a release. You use only what Modules 1 to 12 taught:

- `git status --short`, `git diff`, `git diff --stat`, `git diff --staged`, `git add` with a file name, `git commit -m`
- `git log --oneline`, with `--graph` and a number such as `-3` (as `-1` in the Exercise), and `git ls-files`
- `git switch -c`, `git switch`, `git branch`, `git merge`, `git branch -d` and `-D`
- `git push -u origin`, `git push`, `git pull`, `git tag` and `git push origin v1.0`
- a `.gitignore` and a `README.md` with the four headings (Core concepts 8 and 9)
- on GitHub: an Issue, a pull request with `Closes #n`, line comments and a review, the merge, and a release (Core concepts 10 and 12)
- a link with a full address (Module 1), the DevTools Console (Module 5) and device mode (Module 3), and your test plan, `test-plan.md` (Module 11)

This project starts from the site you built in the packet projects of Modules 1 to 11: five pages (Home, About, Interests, Rate, Contact), one `styles.css`, your scripts, and `test-plan.md` with its five sections, "Every page: navigation and keyboard" last. Every change is committed and pushed, and the live site loads.

A note on what was tested. Every Git command in this part was run for real, with Git 2.43.0 on Linux, in a copy of the sample site. A local repository stood in for GitHub, so pushes and pulls were real, but nothing reached GitHub.com.

The work on the GitHub website (the Issue, the pull request, the review, the merge and the release) could not be done here. The pictures of those pages are drawings, and their button names come from GitHub Docs. The merge commit in the sample was made in the stand-in repository, with the message the lesson shows for a GitHub merge.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds two new files, `.gitignore` and `README.md`. The footer line of all five pages has changed, and `test-plan.md` has two new rows at the end of its "Every page" section. No other file changes, so the live site looks the same except for the new link.

© 2026 559 Solutions LLC

The `.gitignore`. Exactly the three lines from Core concept 8: `.vscode/`, `Thumbs.db` and `*.log`. None of them is a file your pages use.

The README. A `#` heading with your site's name, then the four `##` headings in this order: **What it does**, **Built with**, **What I learned**, **Live site**. One to three sentences under each. What it does names your pages and says that the test plan is in `test-plan.md`. Live site is your address, `https://your-username.github.io/my-site/`. Your pages do not link to either new file, so visitors to the live site see no change from them.

The improvement. The footer of every page ends with a link to your repository. The footer line becomes:

```
<p>Built by hand by Your Name. <a href="https://github.com/your-username/my-site">See the code on GitHub</a></p>
```

Use your own name and username. The link text is **See the code on GitHub** on every page. The link is navy `#1d3557` on the beige footer `#e9dcc9`, 9.1:1. With the keyboard it gets the dark orange ring from Module 4, 3.7:1 on the beige (both checked with the WCAG contrast formula). You write no new CSS.



The new footer link on a phone, reached with Tab only. The ring follows the link when it wraps onto two lines. Nothing scrolls sideways on any of the five pages at 390 px. (real: rendered in Chromium at 390 px)

On GitHub. Issue **Add a link to the code in the footer**, with one or two sentences on why. A branch, `add-footer-link`. A pull request, **Add a GitHub link to the footer**, from `add-footer-link` into `main`. Its description says what changed and how you tested it, and its last line is `Closes #1`. Your own review leaves at least two line comments, and each one is answered by a commit on the branch. You merge it with **Merge pull request**, and the Issue closes.

© 2026 559 Solutions LLC

On your computer, then on GitHub again. `main` brought up to date with `git pull`, the tag `v1.0` on the merge commit and pushed, and a release, **My Site v1.0**, built on that tag.

What a visitor to your repository sees. The README under the file list. A history of small commits with clear messages. A closed Issue linked to a merged pull request. A v1.0 release.

Your Issue and pull request numbers may differ. They are #1 and #2 if your repository has none yet; use the numbers GitHub shows you, everywhere this part says #1 or #2.

The steps

Type every command and every line. Do not paste.

1. **Open `my-site` and check that it is clean.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`. Type `git status`. It must say `nothing to commit, working tree clean`.

   If the only thing listed is an untracked folder such as `.vscode/`, Git says `nothing added to commit but untracked files present` instead. That is fine too; step 3 deals with it. If a page is modified, finish or undo that change first: this project should start from a clean `main`. Then type `git log --oneline -3` and read your last three messages.

2. **Plan on paper.** Write three things. The Issue title and one or two sentences on why a visitor needs the link. Your README's four sections, with the "What I learned" draft from Core concept 9. The five pages whose footer will change. Then write the order of the work: `.gitignore`, README, Issue, branch, commits, pull request, review, merge, pull, tag, release.

3. **The `.gitignore`. Predict:** what will `git status --short` show? Type it. The sample showed one line, `?? .vscode/`, because VS Code had saved a settings file there. Yours may show nothing. In the Explorer, create `.gitignore` at the top of `my-site`, next to `index.html`. Type the three lines, each followed by Enter:

   ```
   .vscode/
   Thumbs.db
   *.log
   ```

   Save. Run `git status --short` again. The sample showed only `?? .gitignore`: the `.vscode/` line was gone. Then type `git ls-files .vscode`. It should print nothing. If it lists a file, that folder was committed in an earlier module; see Problem 6 in Part 4, *Troubleshooting*. Now check and commit:

   ```
   git add .gitignore
   git diff --staged
   ```


© 2026 559 Solutions LLC

```
git commit -m "Add .gitignore"
```

The staged diff shows three `+` lines under `new file mode 100644`. The sample's commit printed `1 file changed, 3 insertions(+)` and `create mode 100644 .gitignore`.

4. **The README.** Create `README.md` next to `index.html`. Type the start:

```
# My Site

## What it does
My personal website: a Home page, an About page with questions people ask me,
an Interests page with a photo gallery, a rating card, and a Contact page whose
form checks what you type. The test plan for every interactive feature is in
test-plan.md.

## Built with
HTML, CSS and JavaScript, written by hand in VS Code. Saved with Git and
published with GitHub Pages.

<!-- Continue alone: the headings What I learned and Live site,
     with your own sentences and your own address. -->
```

Use your own words in What it does. Add the last two sections, then delete the comment. Save, then:

```
git status --short
git add README.md
git diff --staged --stat
git commit -m "Add README"
git push
```

The status line is `?? README.md`. The sample's README was 19 lines, so `--stat` showed `README.md | 19 ++++++++++++++++++` and `1 file changed, 19 insertions(+)`. The push ended with `main -> main`. Open `https://github.com/your-username/my-site`: the README now shows under the file list, with the four headings as headings.

Then check the live site, `https://your-username.github.io/my-site/`, after a few minutes. It looks exactly as before. This build could not test GitHub Pages, so this check is yours: if a page looks different, compare it with the last version before you go on.

5. **Open the Issue (NEW on your site).** On your repository page, click the **Issues** tab, then **New issue**. GitHub Docs (*Creating an issue*) gives the rest. Type the title in the **Title** field: `Add a link to the code in the footer`. In the description, write your one or two sentences from step 2, for example *Visitors who like the site cannot find its code. Add a link to this repository in the footer of every page.* Click **Submit new issue**. Note the number GitHub gives it, such as **#1**.

© 2026 559 Solutions LLC

6. **Make the branch.** Back in the terminal:

```
git switch -c add-footer-link
```

   Git prints `Switched to a new branch 'add-footer-link'`. Every commit until the merge goes on this branch, so `main` and the live site stay as they are.

7. **Change the footer.** Open `index.html` and find the footer line. Change it to the line from the brief, with your own name and username:

```
<p>Built by hand by Your Name. <a href="https://github.com/your-username/my-site">See the code on GitHub</a></p>
```

   Save. Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. The link shows in the footer. Click it: your repository opens on GitHub. Press F5 on the Home page again, then press Tab until the link has the dark orange ring, and press Enter. The same page opens.

   Now change the footer of `about.html`, `interests.html` and `rate.html` the same way. On `contact.html`, make one change on purpose, for the review to find: type the address without `https://`, as `href="github.com/your-username/my-site"`. Do not test the Contact link yet.

8. **Check and commit. Predict:** how many files will `git status --short` list, and how many `+` and `-` lines will `git diff --stat` count? Then:

```
git status --short
git diff --stat
git diff index.html
```

   The sample listed five lines starting with `M`, one per page, and `5 files changed, 5 insertions(+), 5 deletions(-)`. Changing a line shows as one `-` line (the old one) and one `+` line (the new one), so each page has one of each. In `git diff index.html`, read the `-` and `+` pair beside `<footer>`. Then commit all five:

```
git add .
git commit -m "Add GitHub link to footer"
```

   (A warning from `git add` that mentions LF and CRLF is harmless.)

   The sample printed `[add-footer-link fddb8e3] Add GitHub link to footer` and `5 files changed, 5 insertions(+), 5 deletions(-)`. The branch name in the brackets tells you where the commit went.

© 2026 559 Solutions LLC

9. **Break it on purpose: a merge conflict.** A pull request cannot be merged while it has a conflict: "the **Merge pull request** button is deactivated until the conflicts are resolved" (GitHub Docs, *About merge conflicts*). Practice on a throwaway branch, so your real branch is not touched. You will change the Home footer line two ways, as in Exercise step 6:

```
git switch main
git switch -c try-conflict
```

On `main` the footer has no link yet. Change the Home footer line to `<p>Built by hand by Your Name in 2026.</p>` and save. Then `git add index.html` and `git commit -m "Add year to footer"`. **Predict:** which files will merge by themselves, and which one will stop? Then:

```
git merge add-footer-link
git status --short
```

The sample printed:

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

`git status --short` showed `UU index.html` and an `M` in the left column for the other four pages. Those four merged by themselves, because only one side changed them. In `index.html`, the footer now holds the markers:

```
<<<<<<< HEAD
      <p>Built by hand by Your Name in 2026.</p>
=======
      <p>Built by hand by Your Name. <a href="https://github.com/your-
username/my-site">See the code on GitHub</a></p>
>>>>>>> add-footer-link
```

Fix it in VS Code: click **Accept Incoming Change** above the conflict, to keep the branch's line with the link. Save. Then:

```
git add index.html
git commit -m "Merge add-footer-link, keep the link"
git log --oneline --graph -5
```


© 2026 559 Solutions LLC

The graph splits after `Add README` and joins at the merge commit, as in the Exercise. Now throw the practice away:

```
git switch add-footer-link
git branch -d try-conflict
git branch -D try-conflict
```

`-d` refused: `error: the branch 'try-conflict' is not fully merged.` That is Git protecting work. Here you do want to lose it, so `-D` deleted it: `Deleted branch try-conflict (was 04ffdf2).` Type `git branch`: only `add-footer-link` (with the `*`) and `main` are left.

10. **Push the branch.** The branch is new, so it needs `-u` the first time (Core concept 12):

    ```
    git push -u origin add-footer-link
    ```

    Git prints a few lines of progress and your GitHub address. The sample ended with these two lines:

    ```
     * [new branch]      add-footer-link -> add-footer-link
    branch 'add-footer-link' set up to track 'origin/add-footer-link'.
    ```

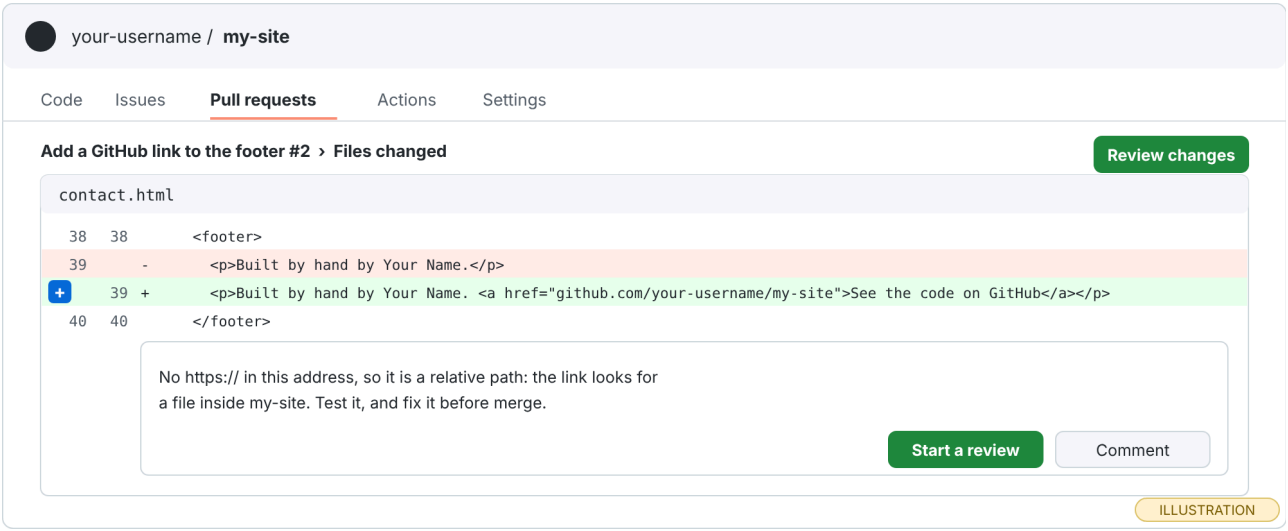
    A push to GitHub may add a few more lines that start with `remote:`. You do not need them.

11. **Open the pull request (NEW on your site).** On your repository page, GitHub Docs (*Creating a pull request*) says to choose your branch in the **Branch** menu and click the yellow **Compare & pull request** banner. Check that **base** is `main` and **compare** is `add-footer-link`. Type the title `Add a GitHub link to the footer`. In the description, write what changed and how you tested it, and end with `Closes #1` on its own line (your Issue's number). Click **Create Pull Request**. Note its number, such as **#2**.

12. **Review your own pull request.** Click **Files changed**. Read every file's diff slowly, as if someone else wrote it. Check three things: the footer line is the same on all five pages; every address starts with `https://`; and the test plan covers the new link. When something is wrong, hover the line number, click the blue comment icon, type a note and click **Start a review** (*Commenting on a pull request*). Leave at least these two:

    - on the `+` line of `contact.html`: the address has no `https://`
    - on the `+` line of `index.html`: `test-plan.md` has no rows for this link yet

© 2026 559 Solutions LLC



The first line comment of the sample's review, pinned to the exact line it is about. Your line numbers may differ.

Then finish the review. Click **Review changes** above the code, write a one-line summary, choose **Comment** and click **Submit review**. Until you do, your notes are "pending and only visible to you" (*Reviewing proposed changes in a pull request*). **Approve** is not for you here: "Pull request authors cannot approve their own pull requests" (same page).

13. **Debug: where does the Contact link go?** Before you fix it, find out what the mistake does. **Predict**, in writing: what will a click on the Contact page's footer link open? Then open `contact.html` from disk, press F5 and click it.

    In the sample, Chromium did not reach GitHub. It could not open the page and reported `ERR_FILE_NOT_FOUND`. To see why, go back to the Contact page, press **F12** (or **Ctrl+Shift+J**) to open DevTools, click the **Console** tab, and type:

    ```
    document.querySelector("footer a").href
    ```

    `.href` gives the full address the browser built from the link. The sample answered `'file:///C:/Users/you/Documents/code559/my-site/github.com/your-username/my-site'`. Without a scheme, the address is a **relative path** (Module 1): "a folder called `github.com` next to this page." That folder does not exist. On the live site, the same link would ask GitHub Pages for a page inside your site that does not exist.

14. **Answer the first comment with a commit.** Add `https://` back to the address in `contact.html`, save, press F5 and click the link: your repository opens. Then:

    ```
    git diff
    git add contact.html
    git commit -m "Fix footer link address on Contact page"
    ```


© 2026 559 Solutions LLC

```
git push
```

The diff shows one `-` line without `https://` and one `+` line with it. This time a plain `git push` works, because `-u` linked the branch in step 10. It ended with `add-footer-link -> add-footer-link`. On GitHub, the pull request updates by itself: "You can continue to commit and push changes in response to the reviews. Your pull request will update automatically" (GitHub Docs, *GitHub flow*). Find your comment on the **Conversation** tab and reply under it with the commit id from your commit line, such as `Fixed in b0b7b79.`

15. **Answer the second comment with a commit.** Open `test-plan.md` and add two rows at the end of the file, which is the end of the "Every page: navigation and keyboard" table (Module 11 kept that section last for this). The first is:

```
| Click See the code on GitHub in the footer of each page | The my-site
repository opens on GitHub | |
```

```
<!-- Continue alone: a second row for the keyboard. Action: Tab to
     the footer link on each page and press Enter. Expected: what the
     ring looks like, and what opens. -->
```

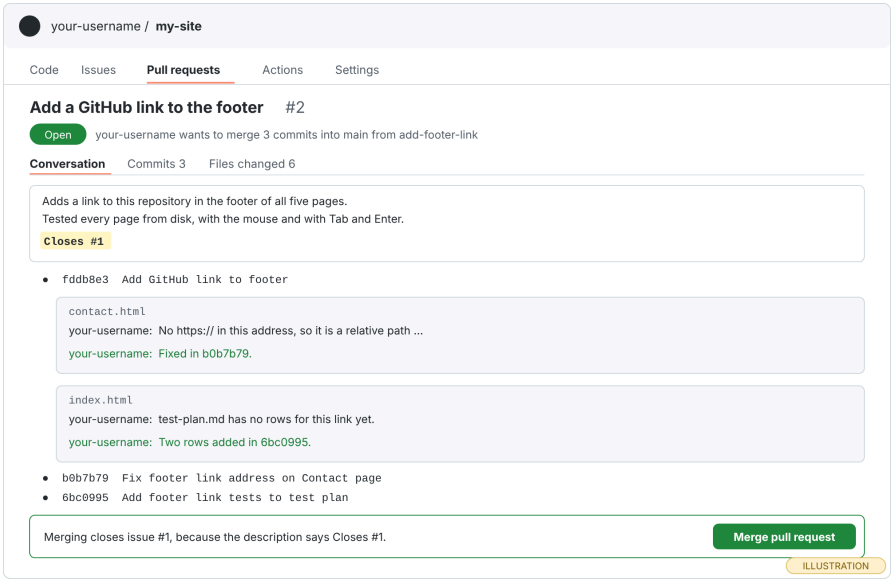
Write the second row alone. Run both rows on all five pages, opened from disk, and fill in Actual with what really happened, such as `Pass on all five pages`. Then:

```
git diff --stat
git add test-plan.md
git commit -m "Add footer link tests to test plan"
git push
```

`--stat` showed `test-plan.md | 2 ++`: two new lines, nothing removed. Reply under your second comment with this commit's id.

16. **Read the pull request again.** Open the **Conversation** tab. Then click **Commits** (three commits in the sample) and **Files changed** (six files: five pages and `test-plan.md`). The Contact line now has `https://`.

© 2026 559 Solutions LLC



The sample's pull request before the merge: each comment is answered by a commit, and the description ends with `Closes #1`.

Modify. Make one more small change of your own on the branch, for example a clearer sentence in the README's What it does. Predict what `git diff --stat` will show, then commit it with a message that finishes "This commit will...", push, and watch the pull request add it.

17. **Explain it, in writing.** In your notebook, write one sentence for each commit on the branch: what it changed and why it is its own commit. Then write why the conflict in step 9 stopped at `index.html` only, and why step 10 needed `-u` but step 14 did not. If you cannot explain one, reread Core concepts 6 and 12.

18. **Test before you merge.** Tick each box.

    - [ ] `git status` says `nothing to commit, working tree clean`, on the branch `add-footer-link`
    - [ ] The footer link opens your repository from all five pages, with the mouse and with Tab and Enter, and has a dark orange ring
    - [ ] Every footer address starts with `https://`; the Contact mistake from step 7 is gone
    - [ ] Nothing scrolls sideways at 390 px (F12, then Ctrl+Shift+M) on any page
    - [ ] `test-plan.md` has the two new rows, with Actual filled in
    - [ ] The pull request goes from `add-footer-link` into `main`, its description ends with `Closes #1`, and every review comment has a reply that names a commit
    - [ ] `try-conflict` is gone (`git branch` lists only `add-footer-link` and `main`)

19. **Merge, pull, tag and release.** Go to Part 3, *Finishing work in Git*.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

In Modules 1 to 11 you finished each project with the short save routine. This time the work is already on GitHub, in a pull request. What is left is the end of the team workflow: merge on GitHub, bring the merge to your computer, check it on the live site, then name the version with a tag and a release. The order matters. A tag goes on the commit you are on, so you tag only after `git pull`.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 11 or in the Project. The sample's numbers below assume the three commits from Project steps 8, 14 and 15; if you made the Modify commit in Project step 16, your counts are one file or one commit higher.

1. **Check the terminal.** The prompt must end in `my-site>`. Type `git status`. It should say `On branch add-footer-link` and `nothing to commit, working tree clean`. If a file is modified, commit it on the branch and push it first, or it stays out of the merge.

2. **Merge the pull request on GitHub (NEW).** Open the pull request and scroll to the bottom of the **Conversation** tab. GitHub Docs (*Merging a pull request*) gives the steps: click **Merge pull request**, accept the commit message GitHub suggests, and click **Confirm merge**. If you see a menu instead, choose **Create a merge commit**. GitHub makes one new commit on `main`, a merge commit like the one in Exercise step 7.

3. **Watch the Issue close (NEW).** Click the **Issues** tab. Issue #1 is no longer in the open list: "When you merge a linked pull request into the default branch of a repository, its linked issue is automatically closed" (GitHub Docs, *Linking a pull request to an issue*). Open it from the closed list and read its timeline. If it is still open, see Problem 5 in Part 4, *Troubleshooting*.

4. **Bring the merge home (NEW).** The merge exists only on GitHub so far. Type:

   ```
   git switch main
   git pull
   ```

   `git switch main` said `Your branch is up to date with 'origin/main'.` It has not asked GitHub yet (Core concept 12), so pull anyway. After a few lines about the download, the sample's `git pull` printed:

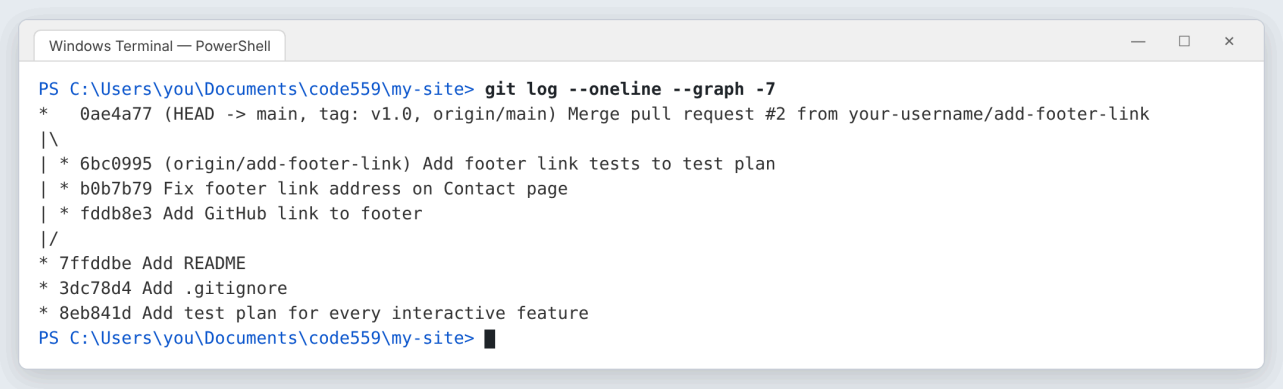
   ```
   Updating 7ffddbe..0ae4a77
   Fast-forward
    about.html     | 2 +-
    contact.html   | 2 +-
    index.html     | 2 +-
    interests.html | 2 +-
   ```


© 2026 559 Solutions LLC

```
 rate.html      | 2 +-
 test-plan.md   | 2 ++
 6 files changed, 7 insertions(+), 5 deletions(-)
```

`Fast-forward` means your `main` simply moved ahead to GitHub's merge commit. Open `index.html` in VS Code: the footer on `main` now has the link.

5. **Read the history (NEW: the graph of a real pull request).** Type `git log --oneline --graph -7`.



The sample's history after step 8 below, so `tag: v1.0` is already on the top line; right after `git pull` that label is not there yet. The three branch commits sit on their own line and join at the merge. `origin/add-footer-link` is your computer's record of the branch on GitHub, which is still there. The bottom commit stands for your last Module 11 commit. (real output, rendered; captured on Linux, with the folder path changed to the course folder. The merge commit was made in a local repository standing in for GitHub.)

Check that the top line says `Merge pull request #2` and `(HEAD -> main, origin/main)`. Git 2.48 and later may also add `origin/HEAD` there (Core concept 12).

6. **Delete the local branch.** Its work is on `main` now:

```
git branch -d add-footer-link
```

The sample printed `Deleted branch add-footer-link (was 6bc0995).` The commits stay; only the name goes. The branch on GitHub can stay too. It does no harm.

7. **Check the live site.** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/`. Run your two new test-plan rows on the live site, on all five pages: a click on the footer link, then Tab and Enter. If a page still shows the old footer, press **Ctrl+Shift+R** to reload without the browser's saved copy (Module 9).

© 2026 559 Solutions LLC

Also run one row of your test plan for each other feature, to see that nothing else changed. The steps in this packet were tested with files opened from disk, not on a live site, so this check is yours.

8. **Tag the version (NEW on your site).** First, check where you are:

   ```
   git log --oneline -1
   ```

   It must show the merge commit, `Merge pull request #2 …`. If it shows `Add README` or a branch commit, stop and see Problem 3 in Part 4. Then:

   ```
   git tag v1.0
   git tag
   git log --oneline -1
   git push origin v1.0
   ```

   `git tag` printed `v1.0`. The log line now has `tag: v1.0` beside `HEAD -> main`. The push ended with `* [new tag] v1.0 -> v1.0`. A plain `git push` would not have sent it (Core concept 10).

9. **Publish the release (NEW on your site).** On your repository page, GitHub Docs (*Managing releases in a repository*) gives the steps. To the right of the file list, click **Releases**, then **Draft a new release**. In **Choose a tag**, pick `v1.0`. In **Release title**, type `My Site v1.0`. In **Describe this release**, write two or three sentences: what the site has, and what is new in this version (the README and the footer link). Click **Publish release**. It looks like the release picture in Core concept 10 of the lesson.

10. **Look at your repository as a visitor.** Open `https://github.com/your-username/my-site` in a private window, so you see what a stranger sees. Check four things: the README under the file list, a v1.0 release beside the files, Issue #1 in the closed Issues with its link to the pull request, and pull request #2 marked as merged. Then, in the terminal, type `git log --oneline` and read your whole history the way a visitor would: small commits, clear messages.

11. **Check that Git is done.** Type `git status`. It should say `On branch main`, `Your branch is up to date with 'origin/main'.` and `nothing to commit, working tree clean`.

> **Checkpoint.** `git status` is clean on `main`. `git log --oneline -1` shows the merge commit with `tag: v1.0`. On GitHub, Issue #1 is closed, pull request #2 is merged, and the release **My Site v1.0** is published. On the live site, the footer link works on all five pages with the mouse and the keyboard, and your test plan still passes.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: six common problems

Each problem has the symptom you see, why it happens, and the fix. Problems 1 to 4 and 6 were made on purpose in a copy of the sample site and run with Git 2.43.0; their messages are quoted as Git printed them. Problem 5 happens only on GitHub.com, which could not be used here, so it follows GitHub Docs. After each fix, run `git status` and read it before you go on.

1. The current branch has no upstream branch

Symptom. You type `git push` on your new branch, and nothing is sent:

```
fatal: The current branch add-footer-link has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin add-footer-link

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
```

Why. The branch exists only on your computer. A plain `git push` does not know which branch on GitHub it belongs to (Core concept 12).

Fix. Push it once with `-u`: `git push -u origin add-footer-link`. `--set-upstream` in the message is the long form of `-u`. After that, a plain `git push` works on this branch.

2. Git refuses to switch branches

Symptom. You type `git switch main` with a page still edited:

```
error: Your local changes to the following files would be overwritten by checkout:
        contact.html
Please commit your changes or stash them before you switch branches.
Aborting
```

Why. The file is different on the two branches, and you have an edit in it that is not committed. Switching would replace the file and lose your edit, so Git stops. You stay on your branch, and nothing changes.

Fix. Run `git diff contact.html` and decide. If the edit belongs to the branch, commit it there (`git add`, `git commit -m "…"`), then switch. If you do not want it, `git restore contact.html` throws it away (Core concept 7), then switch. `stash` is a Git command this course does not teach; you do not need it.

© 2026 559 Solutions LLC

3. The tag is on the wrong commit

Symptom. `git log --oneline -1` after `git tag v1.0` does not show `Merge pull request`. The sample got this two ways:

What happened	What the sample showed
Tagged on `main` before `git pull`	`7ffddbe (HEAD -> main, tag: v1.0, origin/main) Add README`
Ran `git pull` while still on the branch	`git pull` said `Already up to date.`, and the log showed `6bc0995 (HEAD -> add-footer-link, origin/add-footer-link) …`

Why. `git tag` puts the name on the commit you are on (Core concept 10). Before `git pull`, your `main` does not have GitHub's merge commit yet. And `git pull` on a branch brings that branch up to date, not `main`.

Fix. If you have **not** pushed the tag yet, delete it on your computer: `git tag -d v1.0`. The git-tag manual describes `-d` as "Delete existing tags with the given names." The sample printed `Deleted tag 'v1.0' (was 7ffddbe)`. Then `git switch main`, `git pull`, check `git log --oneline -1`, and tag again. If you already pushed the wrong tag, stop and ask for help before you make a release from it.

4. A branch that will not delete

Symptom.

```
error: the branch 'try-conflict' is not fully merged.
If you are sure you want to delete it, run 'git branch -D try-conflict'
```

Why. The branch has commits that no other branch holds. `-d` protects them (Core concept 6). For `try-conflict` in Project step 9 that is expected: its merge was practice. In the sample, `git branch -d add-footer-link` in Finishing step 6 worked without this message, because after `git pull` its commits were on `main`.

A branch you pushed may delete with a warning instead; its commits are safe on GitHub.

Fix. For a practice branch you want to lose, use `-D`, as in Project step 9. For a real branch, do not use `-D`. Check first that its pull request is merged, then run `git switch main` and `git pull`, and try `-d` again.

5. The Issue stayed open after the merge

Symptom. The pull request is merged, but Issue #1 is still in the open list.

Why. GitHub links the two only when the keyword and the number follow its rules (GitHub Docs, *Linking a pull request to an issue*):

© 2026 559 Solutions LLC

Cause	Example
No `#` before the number, or the wrong number	`Closes 1`, or `Closes #2` (the pull request's own number)
The keyword was only in a comment	`Closes #1` typed as a reply on the Conversation tab; the docs name the description (and commit messages) as the places it is read
The pull request went into another branch	base was not `main`; the keywords are read only when it targets the default branch

The keywords that work are close, closes, closed, fix, fixes, fixed, resolve, resolves and resolved, followed by `#` and the Issue's number.

Fix. Close the Issue by hand: open it and click **Close issue** (GitHub Docs, *Closing an issue*). In a comment, write `Done in #2`, so a visitor can still find the work. Next time, check the description before you merge.

6. `.vscode` is still tracked after the `.gitignore`

Symptom. In Project step 3, `git ls-files .vscode` lists a file, such as `.vscode/settings.json`.

Why. "Files already tracked by Git are not affected" by a `.gitignore` (*gitignore*). That folder was committed in an earlier module.

Fix. Take it out of Git, but keep it in your folder (Core concept 8):

```
git rm -r --cached .vscode
git status --short
```

The sample printed `rm '.vscode/settings.json'`. Then the status showed `D .vscode/settings.json` (staged to be removed from Git) and `?? .gitignore`. Add the `.gitignore` and commit both in one commit, with a message such as `Stop tracking .vscode and add .gitignore`. The sample's commit printed `delete mode 100644 .vscode/settings.json`. The file is still in your folder; Git just stops tracking it.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 13. If a line is a 1 or a 2, reread that Core concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can name Git's four places, and read the two columns of git status --short	☐	☐	☐	☐
I can check changes with git diff and git diff --staged, one change per commit	☐	☐	☐	☐
I can keep files out of Git with a .gitignore, and prove it with git status	☐	☐	☐	☐
I can write a README with the four headings for a visitor	☐	☐	☐	☐
I can work on a branch, merge it, delete it with -d, and say when -D fits	☐	☐	☐	☐
I can fix a merge conflict by removing the markers, then git add and git commit	☐	☐	☐	☐
I can push a new branch with -u, and open a pull request with Closes #n	☐	☐	☐	☐
I can review my own pull request and answer each line comment with a commit	☐	☐	☐	☐
I can git pull a merged pull request on main, then tag it and publish a release	☐	☐	☐	☐

Three questions to answer in writing

1. Suppose you had tagged `v1.0` on `main` right after Finishing step 2 (the merge on GitHub), without `git pull`. Which commit would carry the tag in the sample, and what would a visitor who downloads v1.0 be missing?
2. Pick one thing your review found that you had missed in the terminal, or one wrong prediction. What did you expect, what did you see, and what will you do differently?
3. A friend's pull request is merged into `main`, but Issue #4 is still open. The description ends with `This fixes issue 4.` What went wrong, and what should they do now and next time?

© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **Which file is ready to commit?** `styles.css`. Its `M` is in the **left** column, the staging area, so it is staged and goes into the next commit. `index.html` has its `M` in the **right** column, the working folder: it is changed but not staged, so `git commit` leaves it out. To commit it too, run `git add index.html` first. (Core concept 2)

2. **Fast-forward in step 5, a conflict in step 6.** In step 5, `main` had not changed since you made `add-hobby`. Git only had to move `main` forward to the branch's last commit, with no new commit. In step 6, both branches had a new commit, and both changed the same line, `Favorite color`, in two different ways. Git cannot choose between green and orange, so it merges what it can (the hobby line) and stops at that line for you to decide. (Core concept 6; Exercise steps 5 to 7)

3. **What would you test to prove the `.gitignore` works?** Make a file that matches each pattern and one that does not, then read `git status --short`. A file named `debug.log`, a `Thumbs.db` and a file inside `.vscode/` must not appear in the list, while a new `.html` file must.

   Then check that the ignored files are still in the folder (they are: Git stops reporting them, it does not delete them). Last, run `git ls-files .vscode`: it must print nothing, because a file committed before the `.gitignore` is still tracked. In a check run for this packet, a new `notes.log` and a `Thumbs.db` in `my-site` did not appear in `git status --short` at all. (Core concept 8)

Common misconception

"When I commit, my work is saved on GitHub." A commit is saved in the hidden `.git` folder on your computer, in the Commits box of the mental model. Only the `git push` arrow leads from there to GitHub. Until you push, your commit is not on GitHub and not on the live site, and a lost laptop loses it.

`git status` tells you: after a commit without a push, the sample said `Your branch is ahead of 'origin/main' by 1 commit.` and suggested `git push`. The same is true the other way: a merge on GitHub is not on your computer until `git pull`. (Core concepts 1 and 12)

Exercise checks

These are the results of a real run of the Exercise with the lesson's lines. Your commit ids will differ.

© 2026 559 Solutions LLC

- **Step 2.** `?? notes.txt`: untracked. The commit printed `[main (root-commit) …] Add notes` and `1 file changed, 1 insertion(+)`.
- **Step 3.** Both `git diff` and `git diff --staged` showed one context line, `Favorite color: blue`, and one `+` line, `+Hobby: drawing`. (`git diff` showed it before `git add`, `git diff --staged` after.)
- **Step 4.** On `main`, `notes.txt` has only the color line.
- **Step 5.** `Updating …`, `Fast-forward`, and `notes.txt | 1 +`. After `git branch -d add-hobby` (`Deleted branch add-hobby …`), the log has two commits, the newest marked `(HEAD -> main)`.
- **Step 6.** `Auto-merging notes.txt`, `CONFLICT (content): Merge conflict in notes.txt` and `Automatic merge failed; fix conflicts and then commit the result.` `git status --short` shows `UU notes.txt`. The file holds `<<<<<<< HEAD`, `Favorite color: orange`, `=======`, `Favorite color: green`, `>>>>>>> change-color`, then `Hobby: drawing`.
- **Step 7.** After the fix, the commit is a merge commit. `git log --oneline --graph` shows five commits: the line splits after `Add hobby line` into `Change color to orange` on one side and `Change color to green` on the other, and joins at `Merge change-color, keep green`.
- **Step 8.** `git log --oneline -1` shows the merge commit with `(HEAD -> main, tag: v0.1)`.
- **Step 9.** If you committed only on the new branch, the merge is a fast-forward: `main` had not moved since you made the branch. The check run printed `Fast-forward` and `notes.txt | 2 +-`. If you also committed on `main`, Git makes a merge commit and opens the editor for its message.

Self-assessment questions

1. **A tag made before `git pull`.** After Finishing step 2, the merge exists only on GitHub. You are on `add-footer-link`; to tag `main` you would run `git switch main`, which says `up to date` because it has not asked GitHub. The tag lands on the newest commit your `main` has. In the sample, that was `7ffddbe Add README`: a check run showed `7ffddbe (HEAD -> main, tag: v1.0, origin/main) Add README`.

   A visitor who downloads v1.0 gets the README and the `.gitignore`, but not the footer link on any page and not the two test-plan rows: everything from the pull request is missing. That is why you pull first and read `git log --oneline -1` before you tag. (Core concepts 10 and 12; Finishing steps 4 and 8; Problem 3 in Part 4)

2. **Something you missed, or a wrong prediction.** There is no single answer. A good answer quotes what you expected (written before you looked), says exactly what the review, the terminal or the page showed, and names the habit that would have caught it sooner. For example: a missing `https://` that `git diff --stat` could not show, because it counts lines and does not read them; or a conflict you expected in five files that stopped at one.

3. **The friend's Issue #4.** `This fixes issue 4.` has a keyword, `fixes`, but not the form GitHub reads: the keyword followed by `#` and the number, as in `Fixes #4` (GitHub Docs, *Linking a pull request to an issue*). The friend's line does not match that form, which is why the merge did not close the Issue.

© 2026 559 Solutions LLC

Now: open Issue #4, comment with the pull request's number (such as `Done in #5`), and click **Close issue**. Next time: write `Fixes #4` (or `Closes #4`) on its own line in the description, check that the pull request goes into `main`, and read the description once more before you merge. (Core concept 12; Problem 5 in Part 4)

Project checks

- **Step 3.** Before: `?? .vscode/` in the sample (yours may be empty). After the `.gitignore`: only `?? .gitignore`. `git ls-files .vscode` printed nothing. The commit: `1 file changed, 3 insertions(+)`, `create mode 100644 .gitignore`. The whole file, each line ending with Enter:

```
.vscode/
Thumbs.db
*.log
```

- **Step 4, continue alone.** The sample's last two sections, below Built with:

```
## What I learned
How to build pages that work with a keyboard as well as a mouse, and how to
find a bug from the message in the Console. How to keep a project's history
with Git: one change per commit, branches, and pull requests.

## Live site
https://your-username.github.io/my-site/
```

  With them the README is 19 lines: `README.md | 19 ++++++++++++++++++++`. The push ended with `main -> main`.

- **Step 7, continue alone.** The footer line in `about.html`, `interests.html` and `rate.html` is the same line as in `index.html`, letter for letter:

```
<p>Built by hand by Your Name. <a href="https://github.com/your-username/my-
site">See the code on GitHub</a></p>
```

  `contact.html` gets the planted mistake: `href="github.com/your-username/my-site"`.

- **Step 8.** Five `M` lines, `5 files changed, 5 insertions(+), 5 deletions(-)`. In `git diff index.html`, the `-` line is `<p>Built by hand by Your Name.</p>` and the `+` line is the new footer line, with the unchanged `<footer>` and `</footer>` lines around them.

- **Step 9.** The conflict stops at `index.html` only (`UU index.html`); the other four pages show `M` in the left column. After **Accept Incoming Change** and the commit, `git log --oneline --graph -5` shows the split after `Add README` and the join at `Merge add-footer-link, keep the link`. `-d` refuses with `not fully merged`; `-D` prints `Deleted branch try-conflict (was …)`.

© 2026 559 Solutions LLC

- **Step 10.** `* [new branch] add-footer-link -> add-footer-link` and `branch 'add-footer-link' set up to track 'origin/add-footer-link'.`
- **Step 13.** The click does not reach GitHub: Chromium reports `ERR_FILE_NOT_FOUND`. The Console line gives the full address of a file inside `my-site`, ending in `/my-site/github.com/your-username/my-site`.
- **Step 14.** The diff has one `-` line without `https://` and one `+` line with it; `1 file changed, 1 insertion(+), 1 deletion(-)`.
- **Step 15, continue alone.** The sample's two rows, with Actual filled in after the run:

  ```
  | Click See the code on GitHub in the footer of each page | The my-site repository opens on GitHub | Pass on all five pages |
  | Tab to the footer link on each page and press Enter | A dark orange ring on the link; Enter opens the repository | Pass on all five pages |
  ```

  `test-plan.md | 2 ++`, `1 file changed, 2 insertions(+)`.
- **Step 18.** If you followed the brief, every box is ticked. The most common misses: a page whose footer still has the old line (`git diff --stat` on the branch lists fewer than five pages), and a `Closes` line with the wrong number.

NEXT

Module 13: Capstone.
Free lesson: 559solutions.com/code559/module-13-capstone/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC